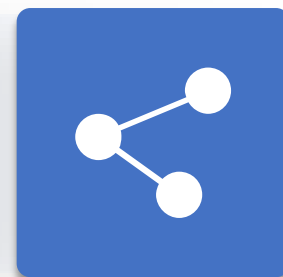




# 人工智能引论

## 第5章 对抗搜索和博弈

授课教师：李静瑶





# 录

1

博弈论

2

博弈中的优化决策

3

启发式 $\alpha$ - $\beta$ 树搜索

4

蒙特卡罗树搜索

5

随机博弈



## 对抗搜索

- 竞争环境：两个或两个以上的智能体具有相互冲突的目标
  - 博弈：国际象棋、围棋、扑克
  - 体育比赛：冰球、足球
- 人工智能领域的博弈：确定性、完全可观测、双人、轮流~~的零和~~博弈
  - 一方收益的增加，必然意味着另一方收益的等量减少，总收益为零
- 博弈问题通常难以求解
  - 国际象棋平均分支因子约为35，一盘棋一般每个玩家走50步，节点数超过 $10^{40}$



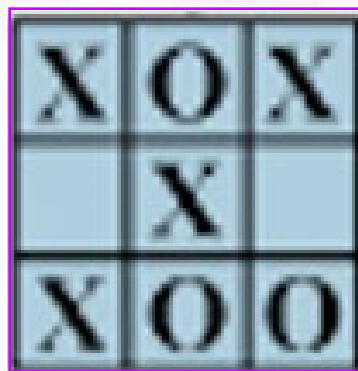
## 双人零和博弈

- 两名玩家：MAX先移动，MIN后移动，二者轮流参与
- $S_0$ ：初始状态，指定博弈开始时如何设置
- To-Move(s)：在状态s下，轮到其移动的参与者
- Actions(s)：在状态s下，全体合法移动的集合
- Result(s, a)：转移模型，定义状态s下执行动作a所产生的结果状态
- Is-Terminal(s)：终止测试，博弈结束时返回真 → 终止状态
- Utility(s, p)：效用函数，定义终止状态s下参与者p得到的最终的数值收益（游戏规则决定，例如国际象棋中 1/0/0.5）

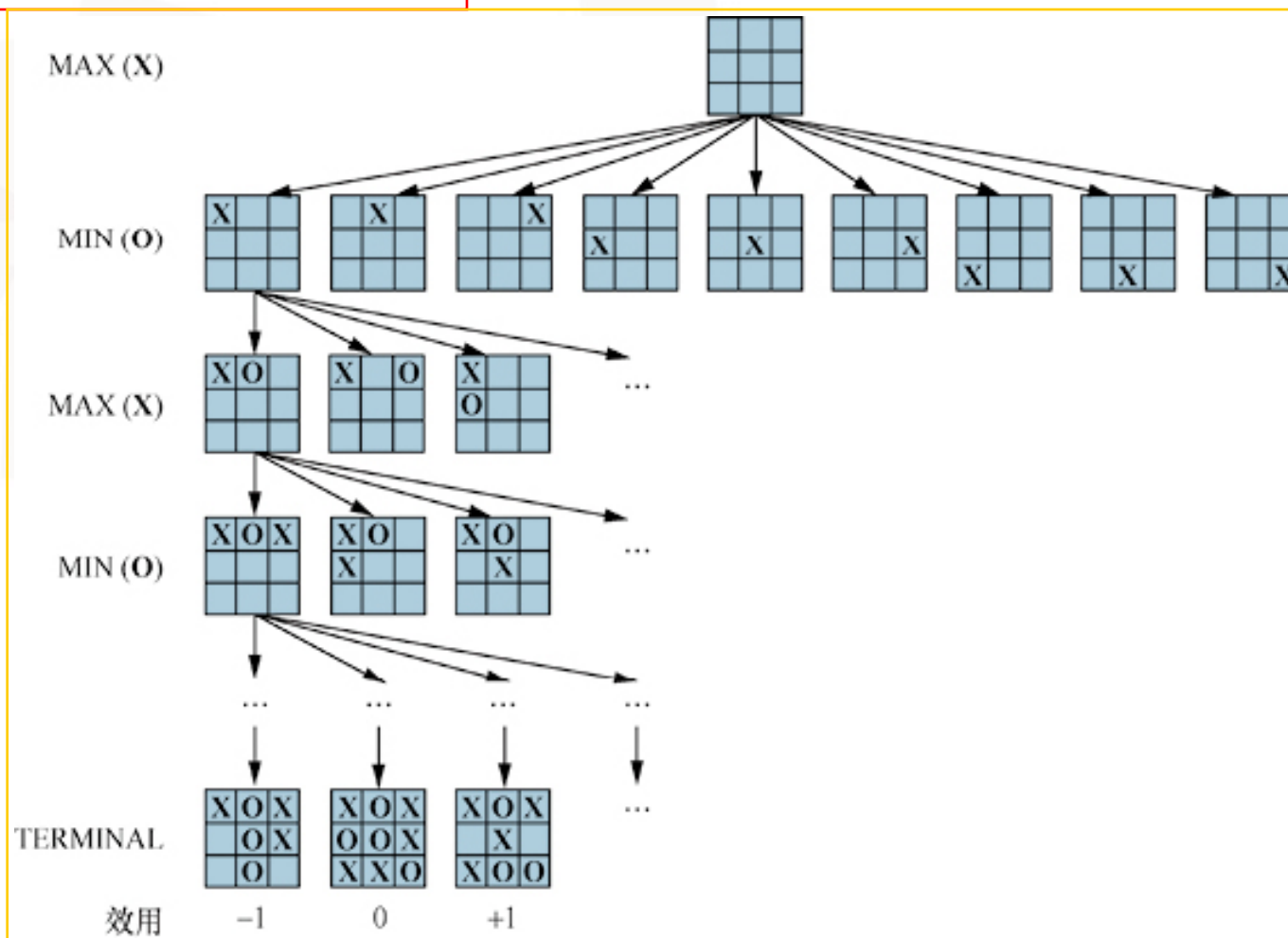


## 例：井字棋游戏

- 一个 $3 \times 3$ 的棋盘，共9个位置
- 两名玩家轮流在空位置填写X或O，直到一名玩家的标记占领一行、一列、一条对角线或无空位置
- 占领一行、一列、一条对角线的玩家获胜



## 井字棋游戏的部分博弈树



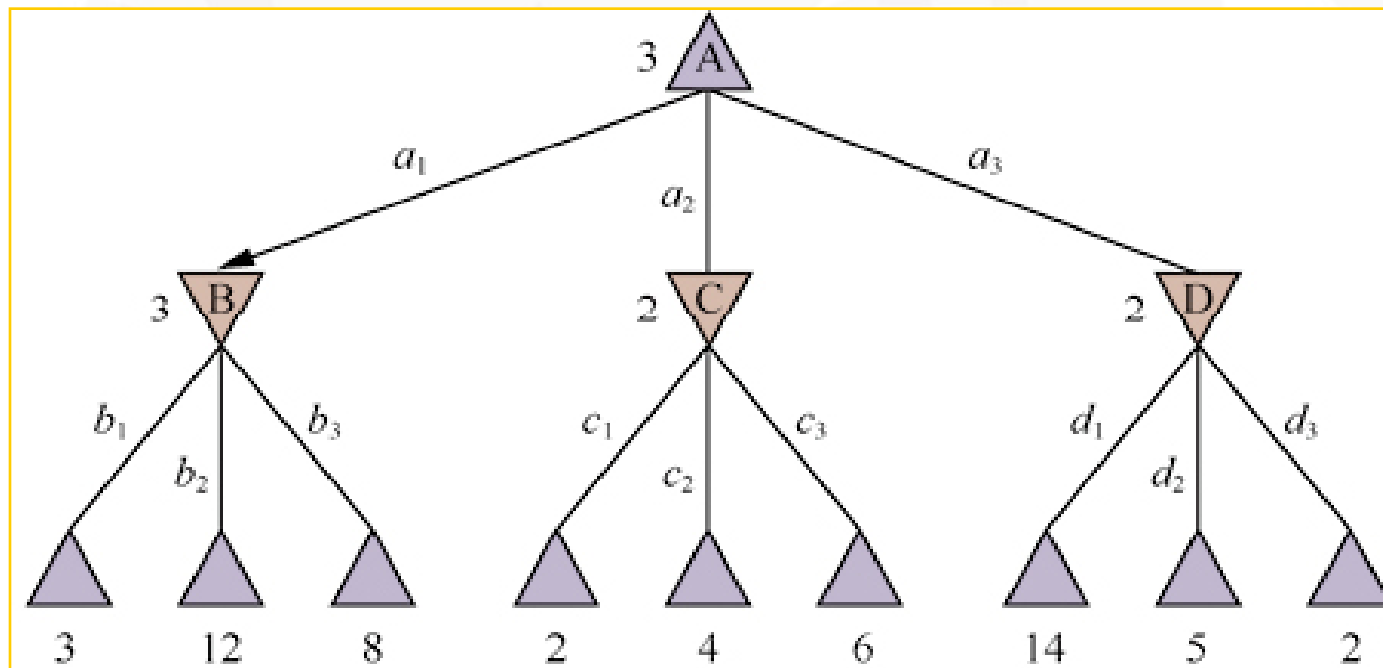


- 条件规划：MAX想找到通往胜利的动作序列，但是MIN不希望MAX获胜，所以MAX的策略必须是一个条件规划，即随机应变策略，指定对MIN的每个可能移动的响应
- 极小极大搜索

## 例：两层的博弈树

MAX

MIN





- 极小极大值（倒推值）：假设从该状态到博弈结束两个参与者都以最优策略移动，到达终止状态对于MAX的效用值

- 极小极大决策

- $\text{MinMax}(s) =$

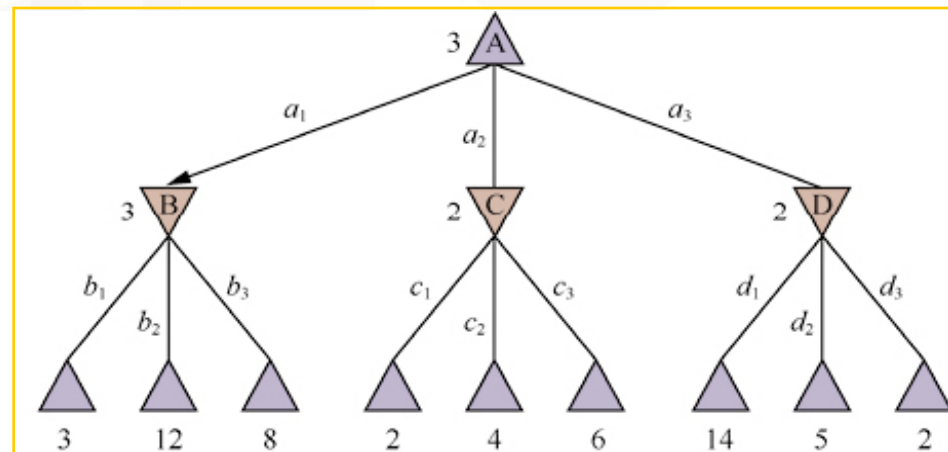
▸  $\text{Utility}(s, \text{Max}), \text{ IF } \text{Is-Terminal}(s)$

▢  $\text{Max}_{a \in \text{Actions}(s)} \text{MinMax}(\text{Result}(s,a)), \text{ IF } \text{To-Move}(s) = \text{Max}$

▸  $\text{Min}_{a \in \text{Actions}(s)} \text{MinMax}(\text{Result}(s,a)), \text{ IF } \text{To-Move}(s) = \text{Min}$

MAX

MIN



**function** MINIMAX-SEARCH(*game, state*) **returns** 一个动作

player  $\leftarrow$  game.TO-MOVE(*state*)

value, move  $\leftarrow$  MAX-VALUE(*game, state*)

**return** move

**function** MAX-VALUE(*game, state*) **returns** 一个(*utility, move*)对

**if** game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state, player*), null

v, move  $\leftarrow -\infty$ , null

**for each** a **in** game.ACTIONS(*state*) **do**

v2, a2  $\leftarrow$  MIN-VALUE(*game, game.RESULT(state, a)*)

**if** v2 > v **then**

v, move  $\leftarrow$  v2, a

**return** v, move

**function** MIN-VALUE(*game, state*) **returns** 一个(*utility, move*)对

**if** game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state, player*), null

v, move  $\leftarrow +\infty$ , null

**for each** a **in** game.ACTIONS(*state*) **do**

v2, a2  $\leftarrow$  MAX-VALUE(*game, game.RESULT(state, a)*)

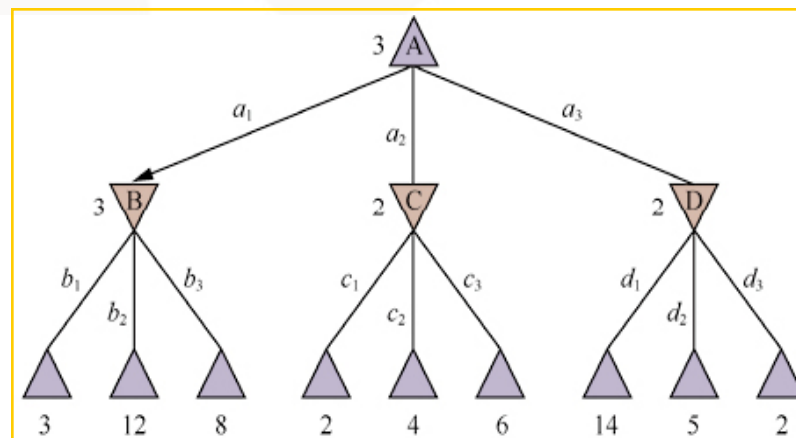
**if** v2 < v **then**

v, move  $\leftarrow$  v2, a

**return** v, move

MAX

MIN





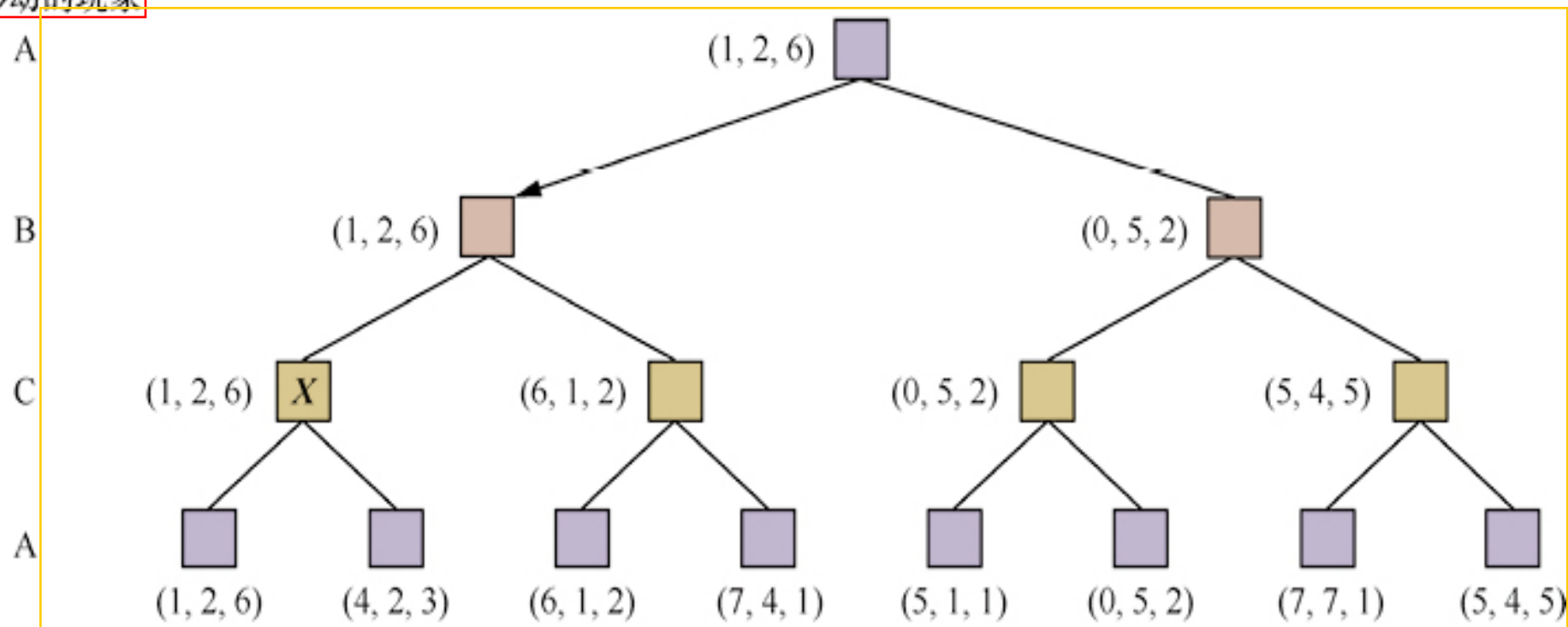
- 最优移动：假定对手移动是为了使效用值最小的前提下，使终止状态效用值最大的移动
- 当MIN不按照最优策略移动时，MAX至少会和MIN一样好，甚至可能更好
- 极小极大搜索得到的策略不一定是最好的



多人博弈

$\langle V_A, V_B, V_C \rangle$

轮到移动的玩家





## 多人博弈

- 随着游戏的进行，联盟不断建立和瓦解
- 联盟是每个参与者按照最优策略行动的自然结果
- 联盟的即时优势和失去信任带来的长期劣势（多智能体决策）



## 极小极大算法

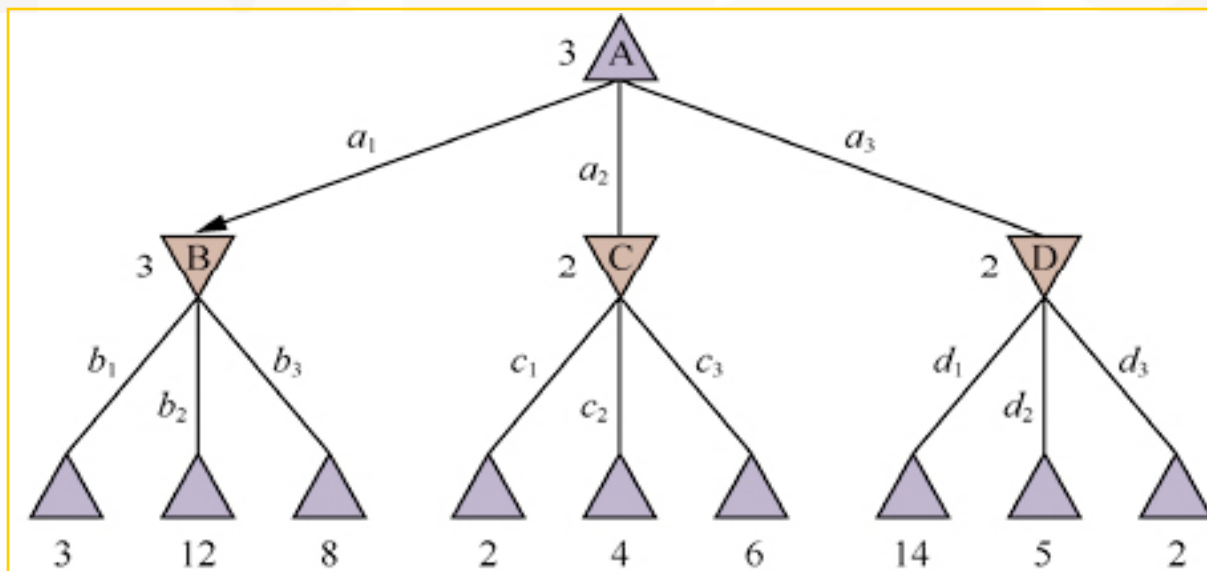
- $m$ 为搜索树的最大深度,  $b$ 为平均分支因子
- 时间复杂度:  $O(b^m)$
- 缺点: 指数级复杂度, 无法应用于复杂博弈
  - 国际象棋:  $m \approx 80, b \approx 35, 35^{80} \approx 10^{123}$

$\alpha$ - $\beta$ 剪枝

- 消除对结果没有影响的树的大部分，从而不需要检查所有状态就能计算出正确的极小极大决策

MAX

MIN



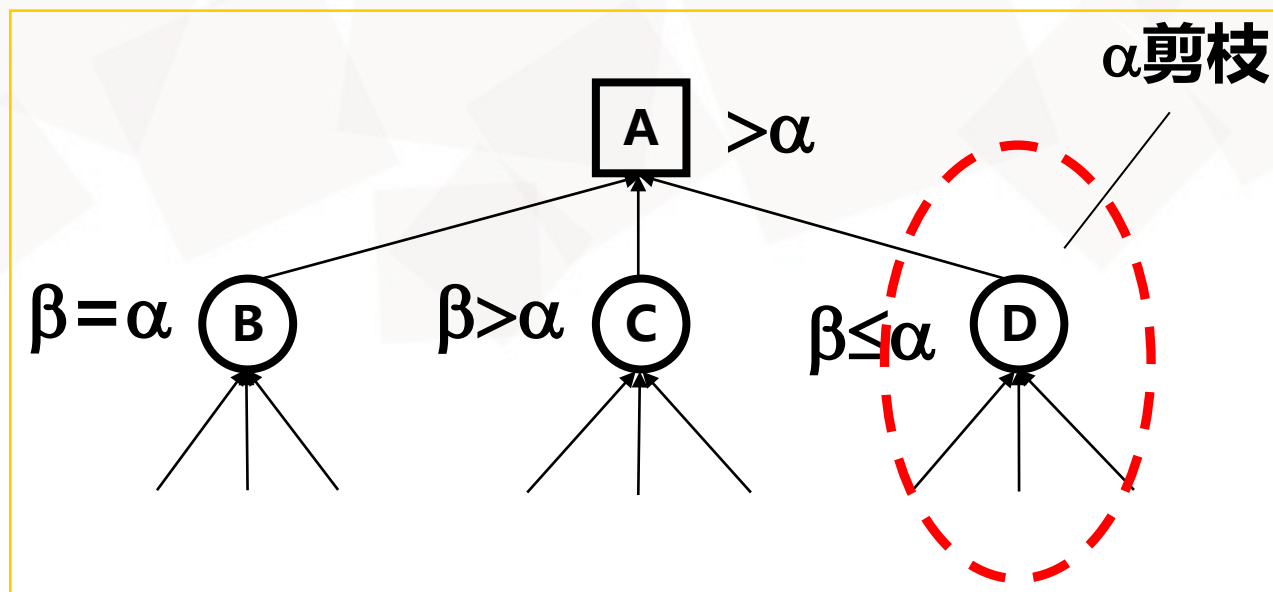


$\alpha$  剪枝

- 设MAX节点的下限为 $\alpha$ ，则其所有的MIN子节点中，其评估值的 $\beta$ 上限小于等于 $\alpha$ 的节点，其以下部分的搜索可以停止，即对这部分节点进行了 $\alpha$ 剪枝

MAX节点

MIN节点



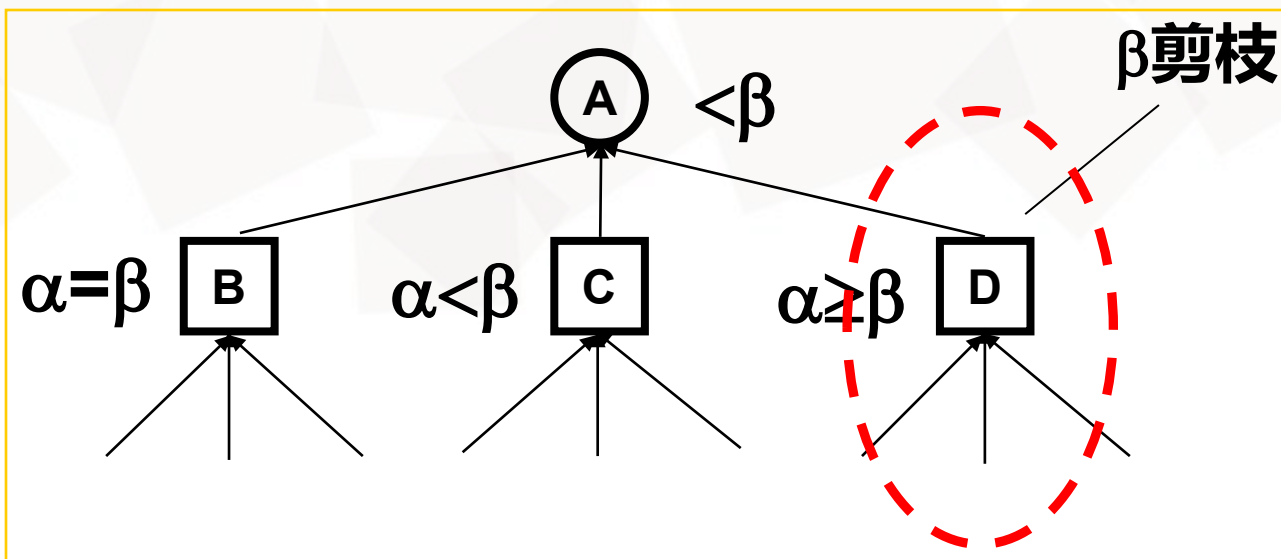


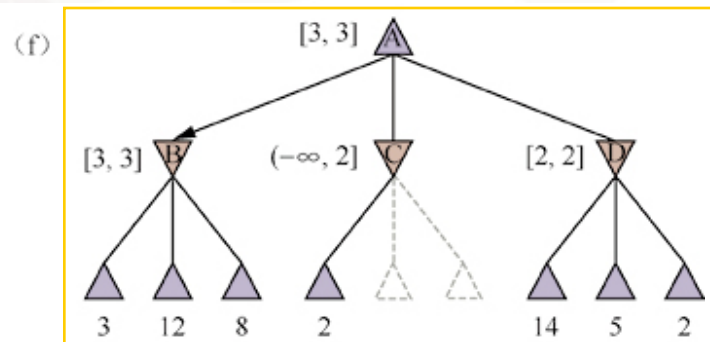
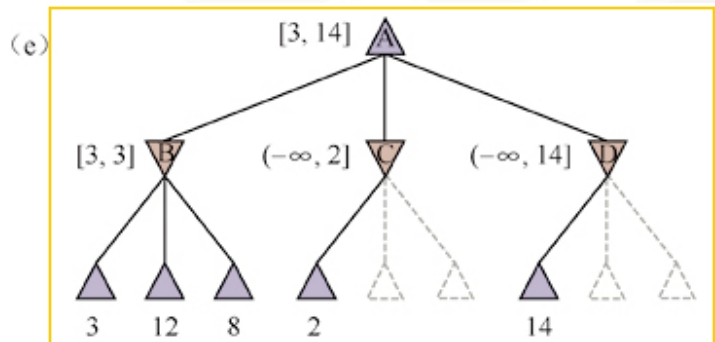
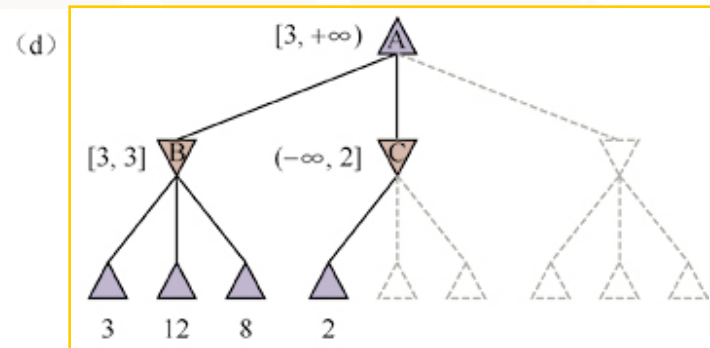
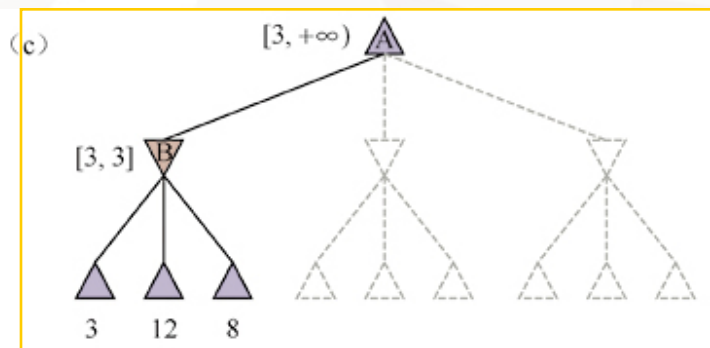
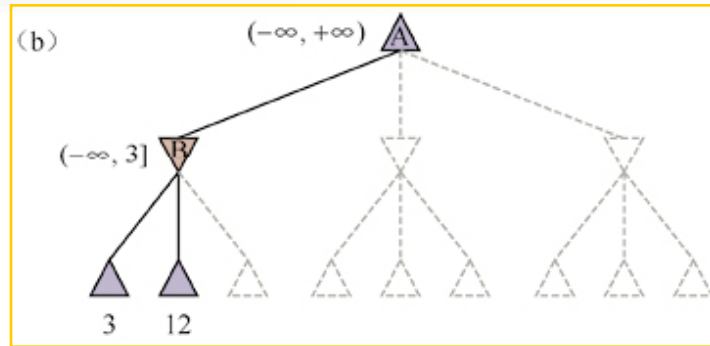
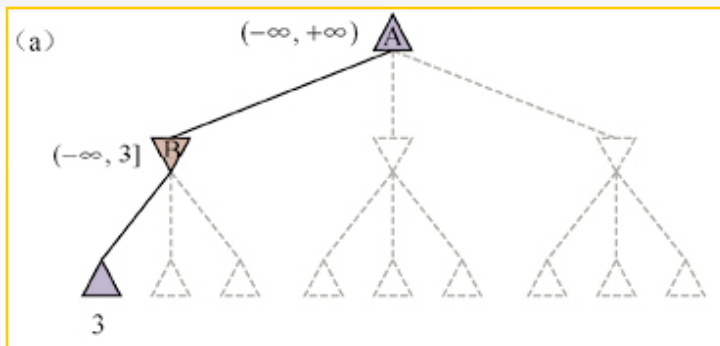
## $\beta$ 剪枝

- 设MIN节点的上限为 $\beta$ ，则其所有的MAX子节点中，其评估值的 $\alpha$ 下限大于等于 $\beta$ 的节点，其以下部分的搜索可以停止，即对这部分节点进行了 $\beta$ 剪枝

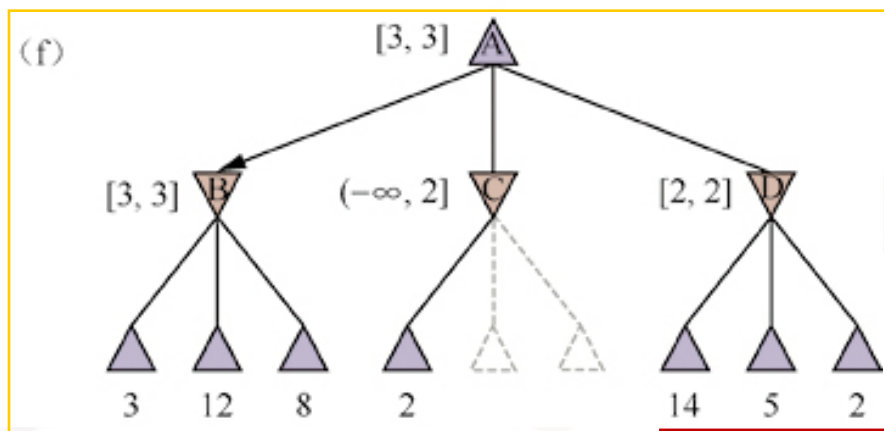
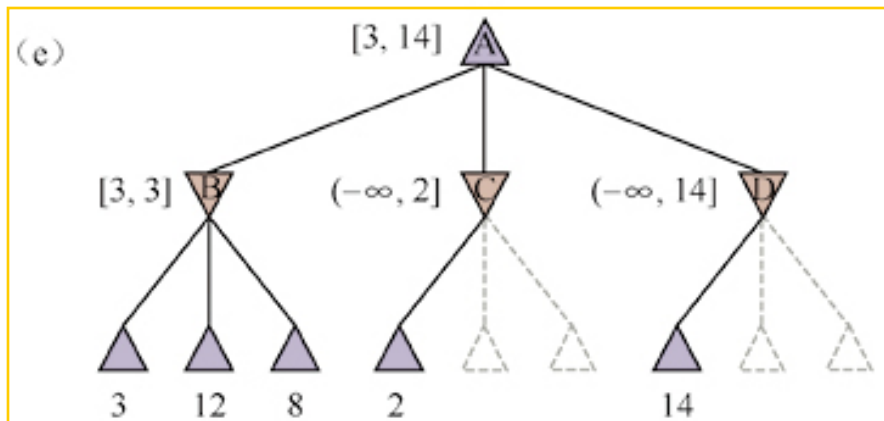
MIN节点

MAX节点





## $\alpha$ - $\beta$ 剪枝依赖于移动顺序



# 博弈中的优化决策



**function** ALPHA-BETA-SEARCH(*game*, *state*) **returns** 一个动作

player  $\leftarrow$  game.To-Move(*state*)

value, move  $\leftarrow$  MAX-VALUE(*game*, *state*,  $-\infty$ ,  $+\infty$ )

**return** move

**function** MAX-VALUE(*game*, *state*,  $\alpha$ ,  $\beta$ ) **returns** 一个(utility, move)对

**if** game.Is-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null

$v$ , move  $\leftarrow -\infty$ , null

**for each**  $a$  **in** game.ACTIONS(*state*) **do**

$v2$ ,  $a2 \leftarrow$  MIN-VALUE(*game*, game.RESULT(*state*,  $a$ ),  $\alpha$ ,  $\beta$ )

**if**  $v2 > v$  **then**

$v$ , move  $\leftarrow v2$ ,  $a$

$\alpha \leftarrow$  MAX( $\alpha$ ,  $v$ )

**if**  $v \geq \beta$  **then return**  $v$ , move

**return**  $v$ , move

**function** MIN-VALUE(*game*, *state*,  $\alpha$ ,  $\beta$ ) **returns** 一个(utility, move)对

**if** game.Is-TERMINAL(*state*) **then return** game.UTILITY(*state*, player), null

$v$ , move  $\leftarrow +\infty$ , null

**for each**  $a$  **in** game.ACTIONS(*state*) **do**

$v2$ ,  $a2 \leftarrow$  MAX-VALUE(*game*, game.RESULT(*state*,  $a$ ),  $\alpha$ ,  $\beta$ )

**if**  $v2 < v$  **then**

$v$ , move  $\leftarrow v2$ ,  $a$

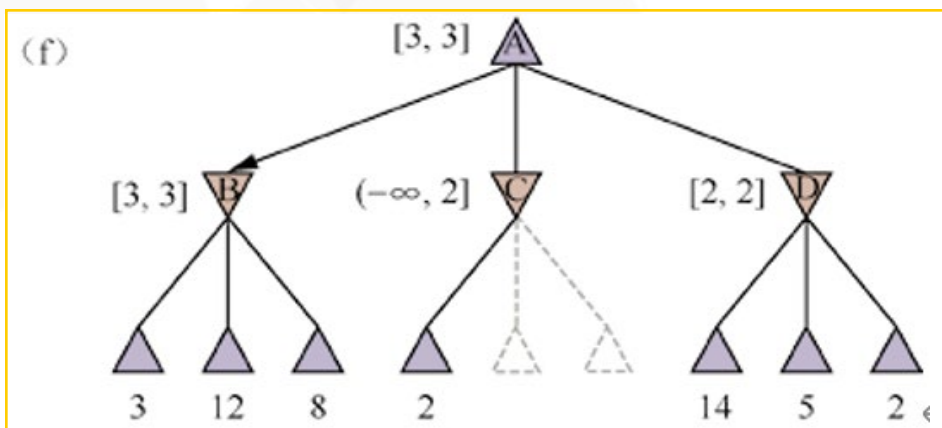
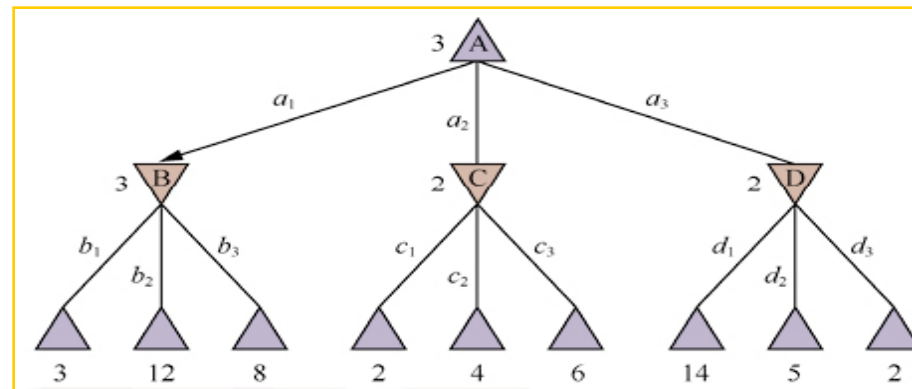
$\beta \leftarrow$  MIN( $\beta$ ,  $v$ )

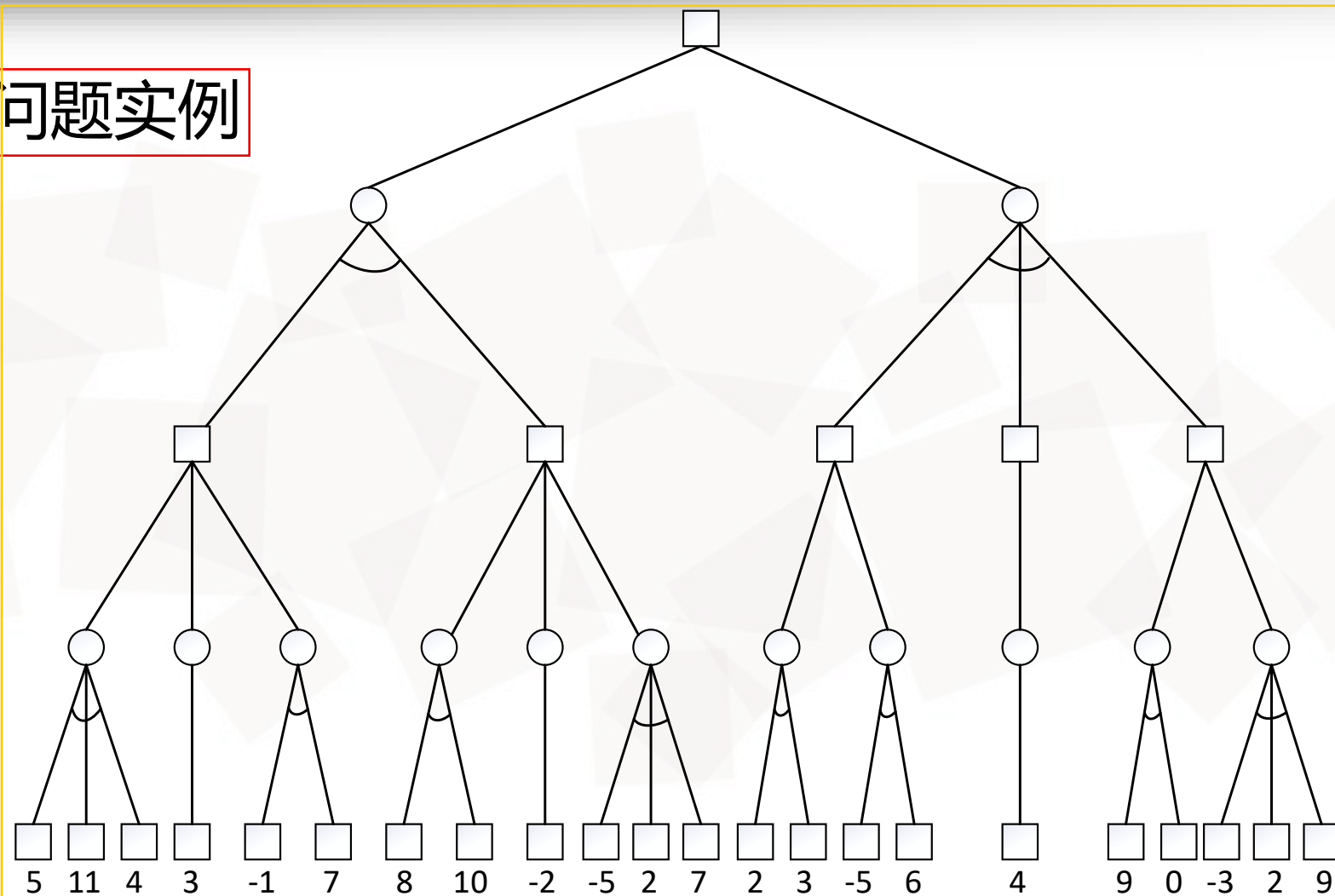
**if**  $v \leq \alpha$  **then return**  $v$ , move

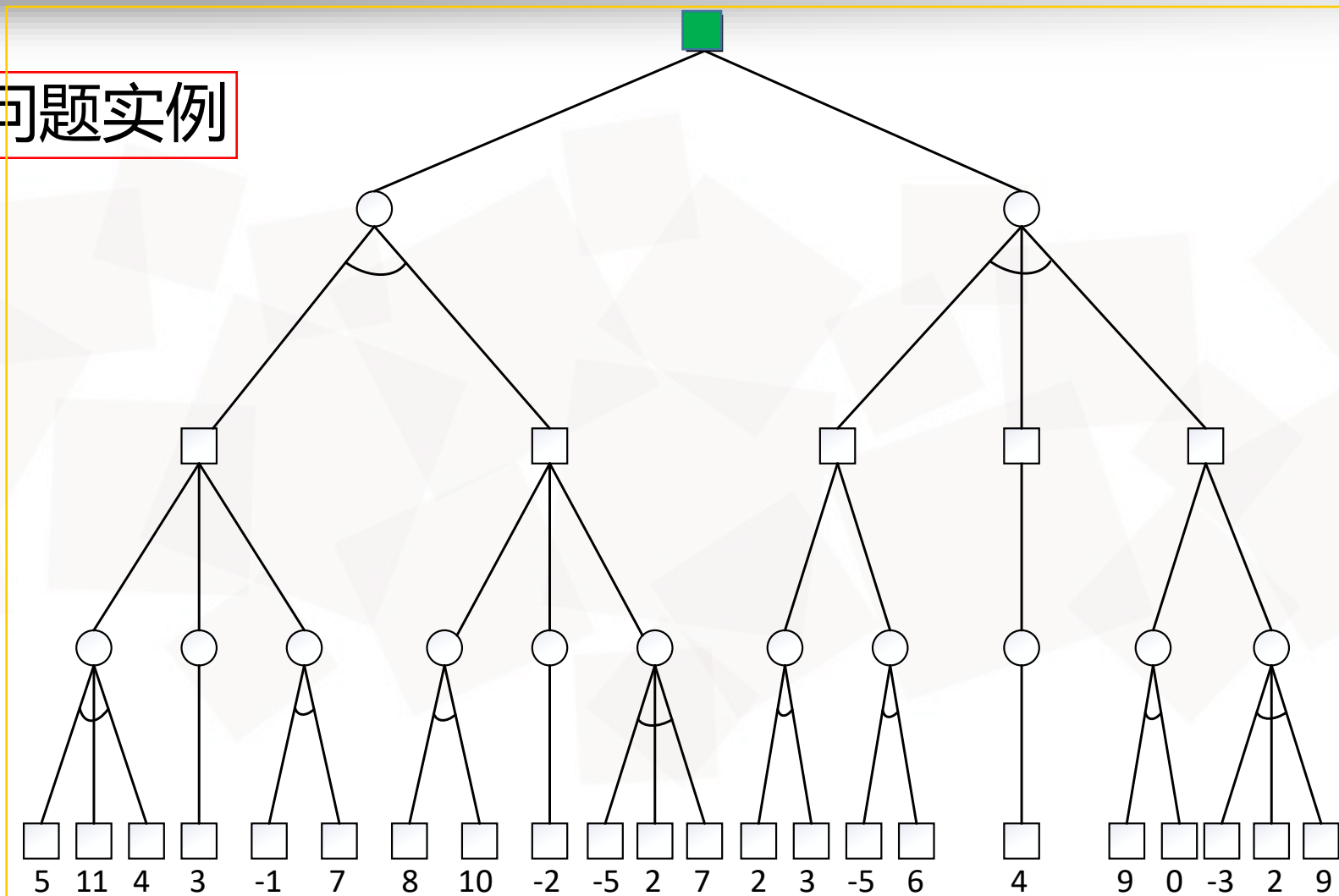
**return**  $v$ , move

MAX

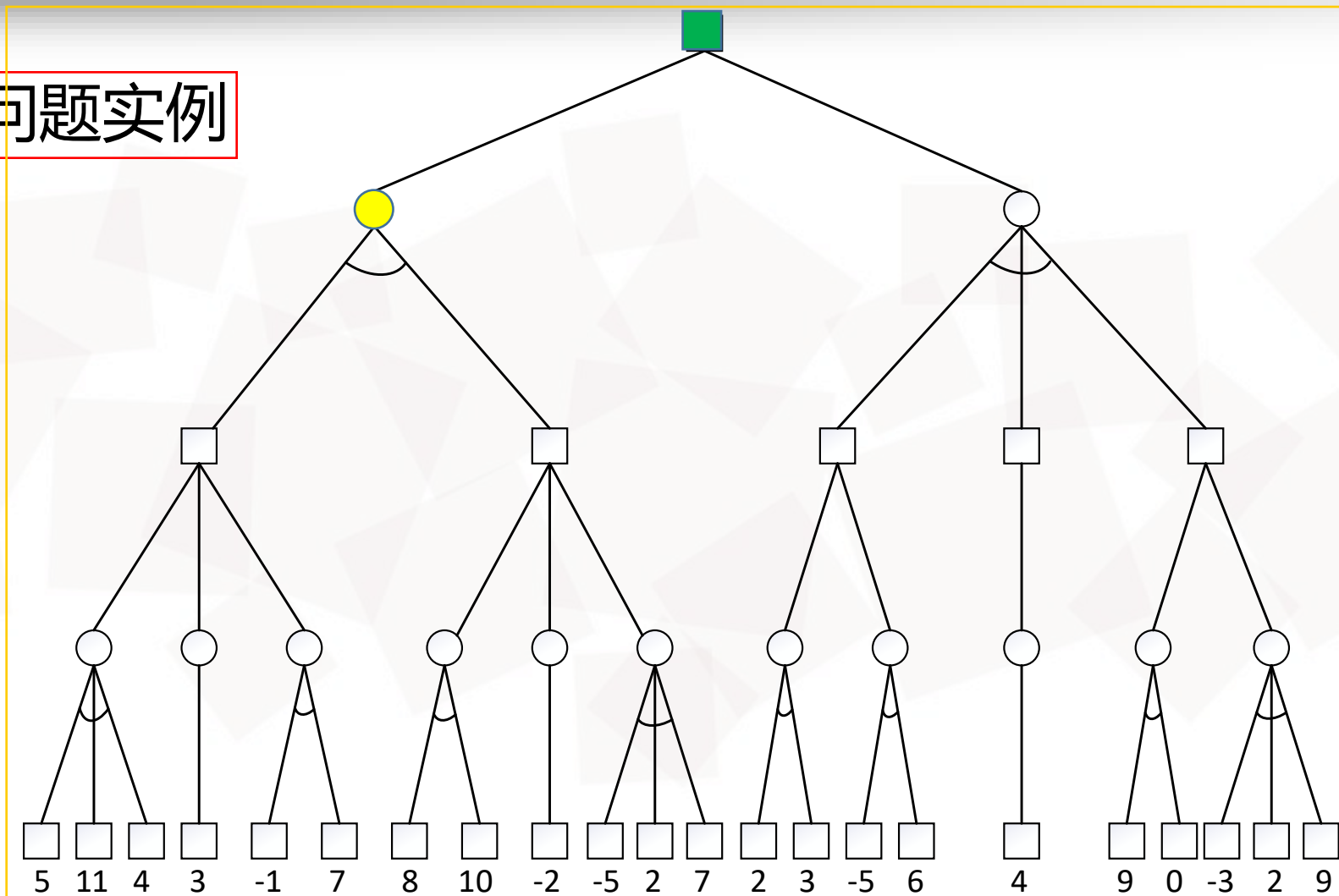
MIN



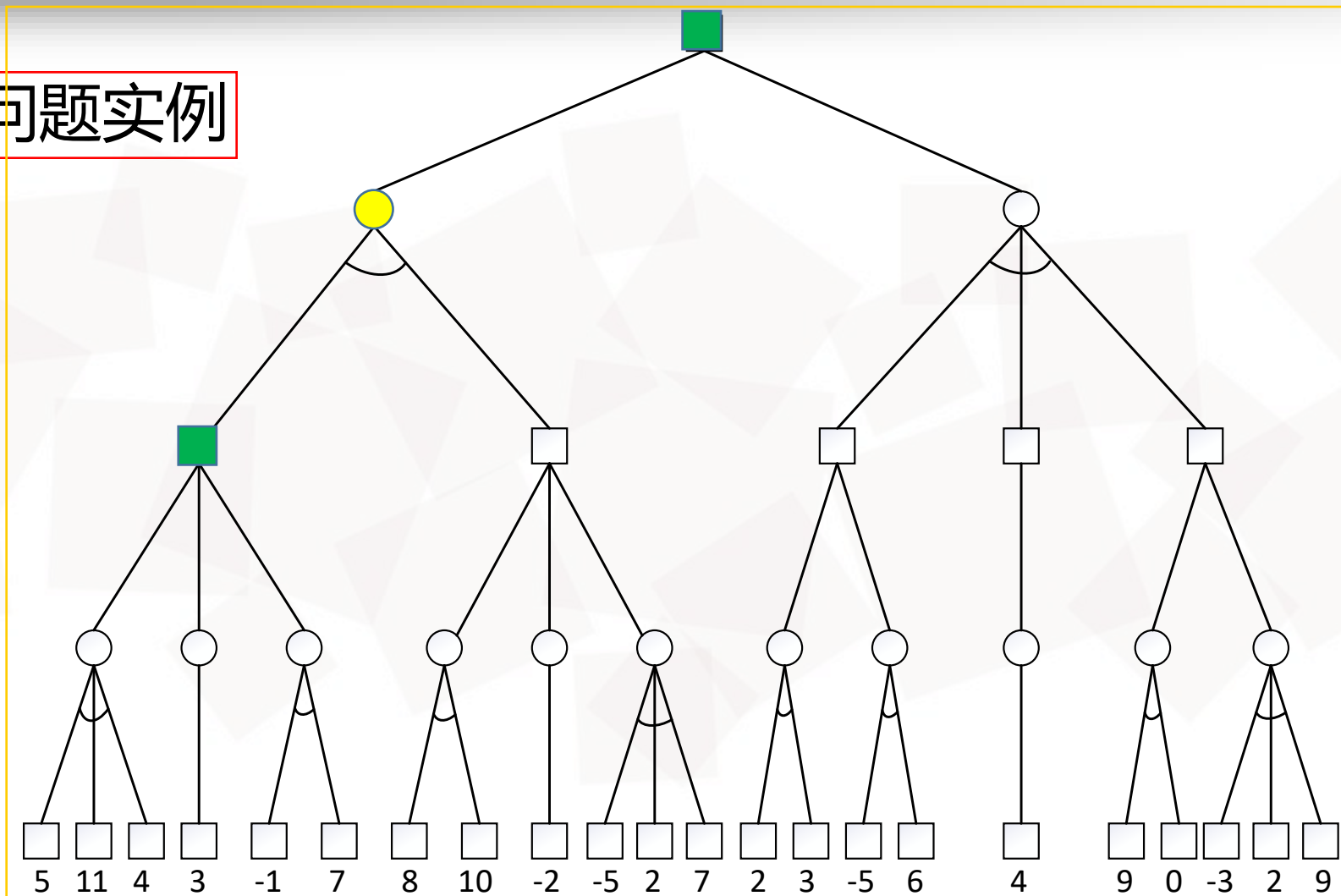
$\alpha$ - $\beta$ 剪枝问题实例

$\alpha$ - $\beta$ 剪枝问题实例

## $\alpha$ - $\beta$ 剪枝问题实例





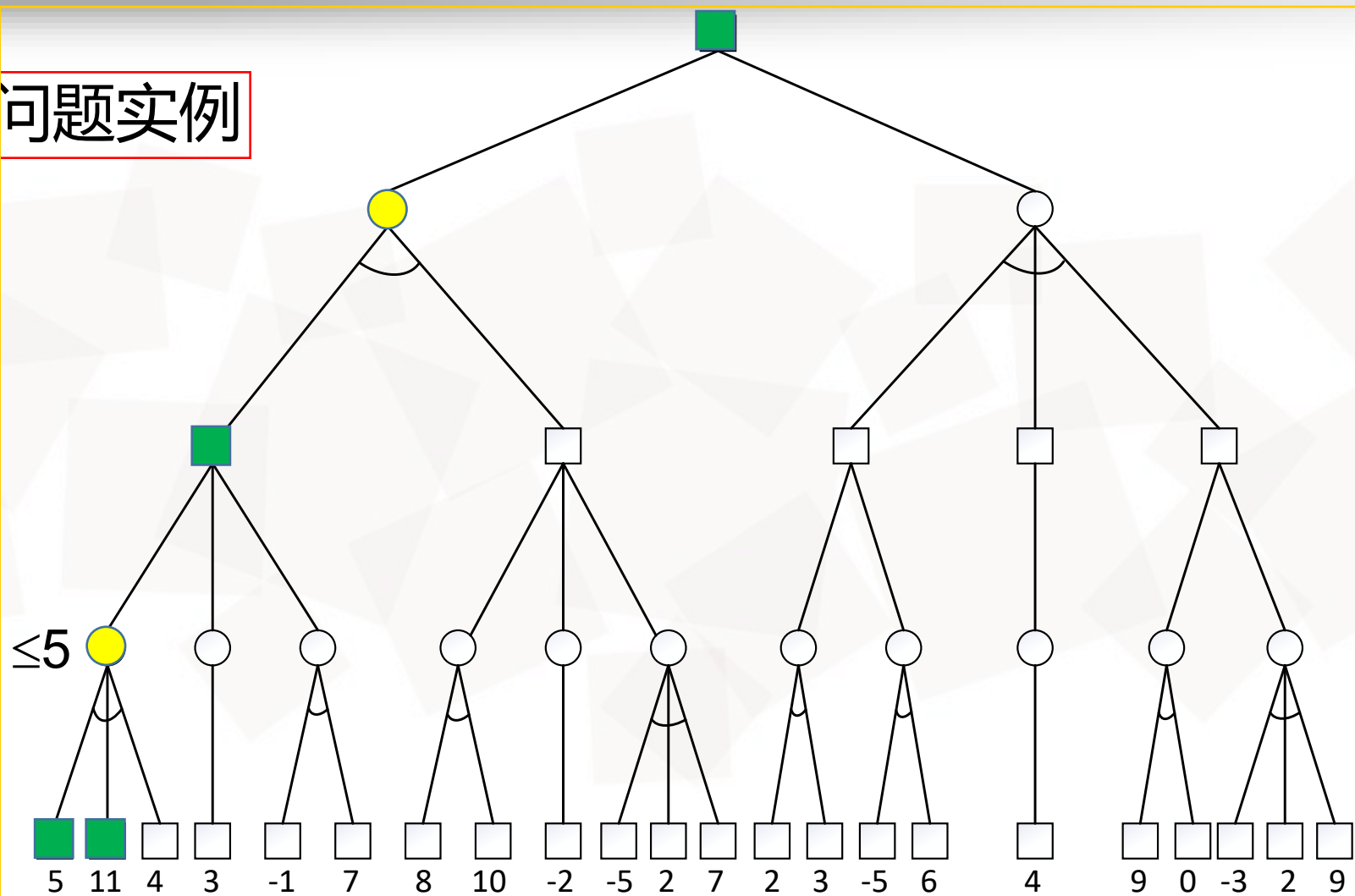
$\alpha$ - $\beta$ 剪枝问题实例



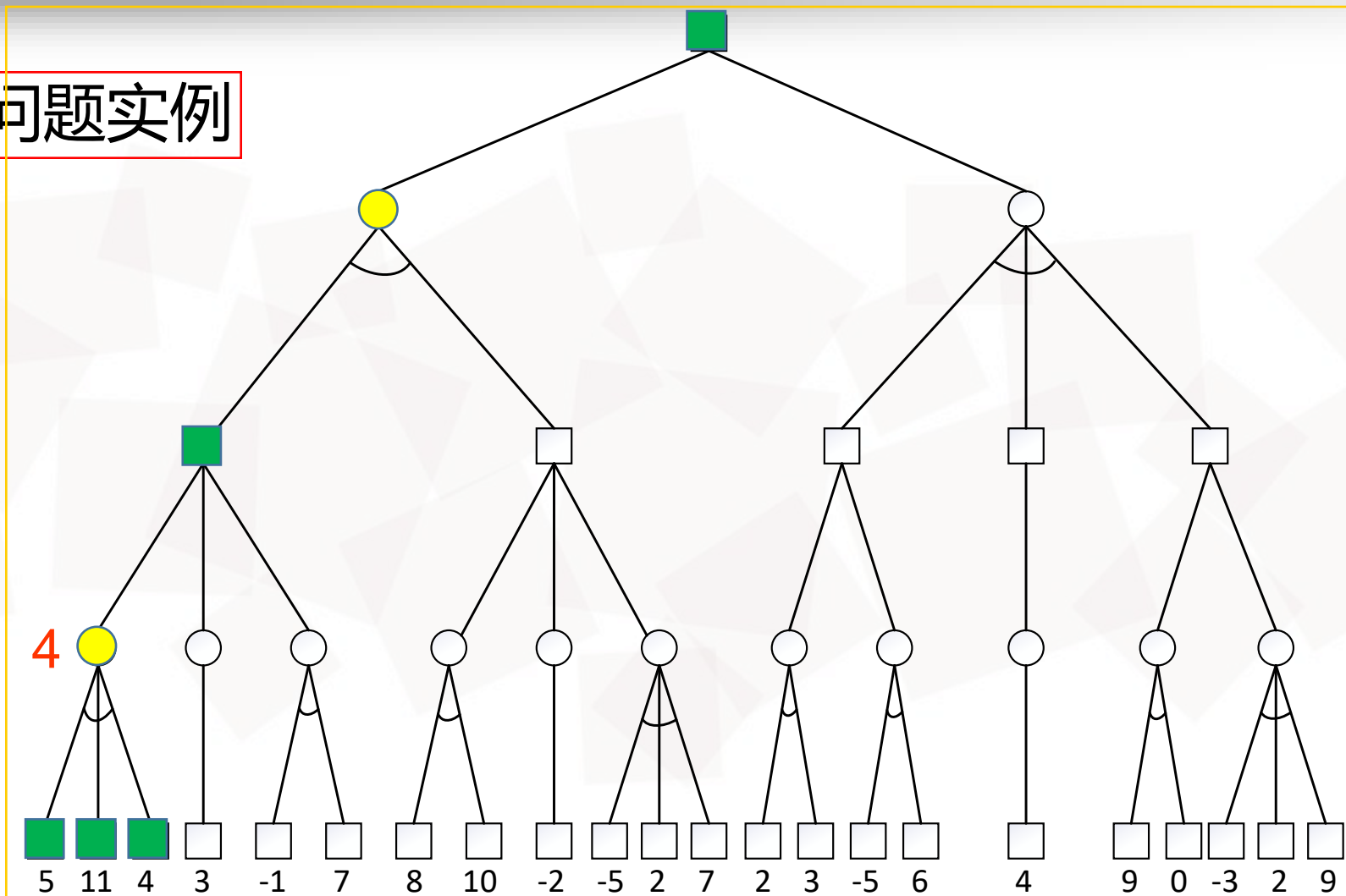


## $\alpha$ - $\beta$ 剪枝问题实例

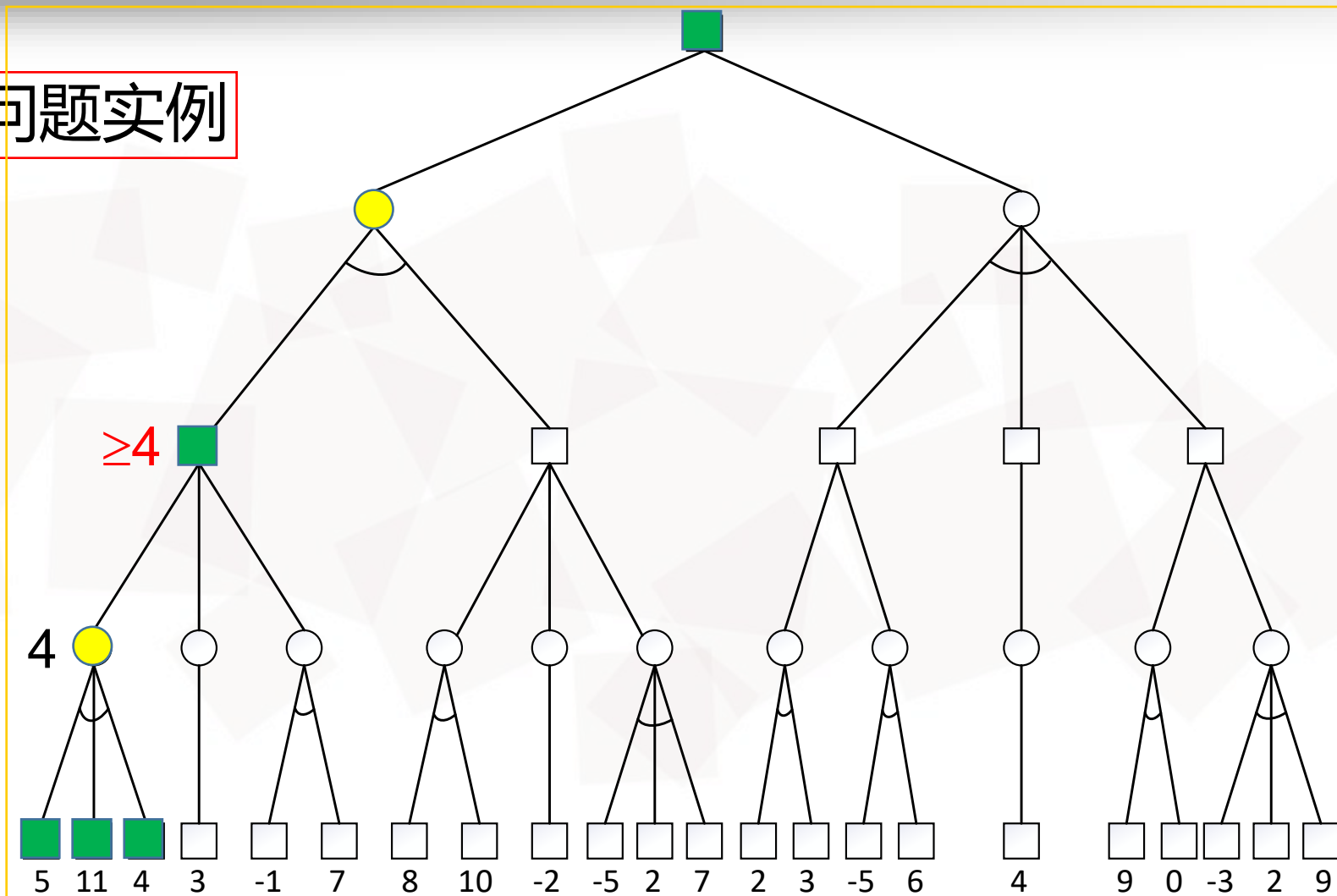


$\alpha$ - $\beta$ 剪枝问题实例

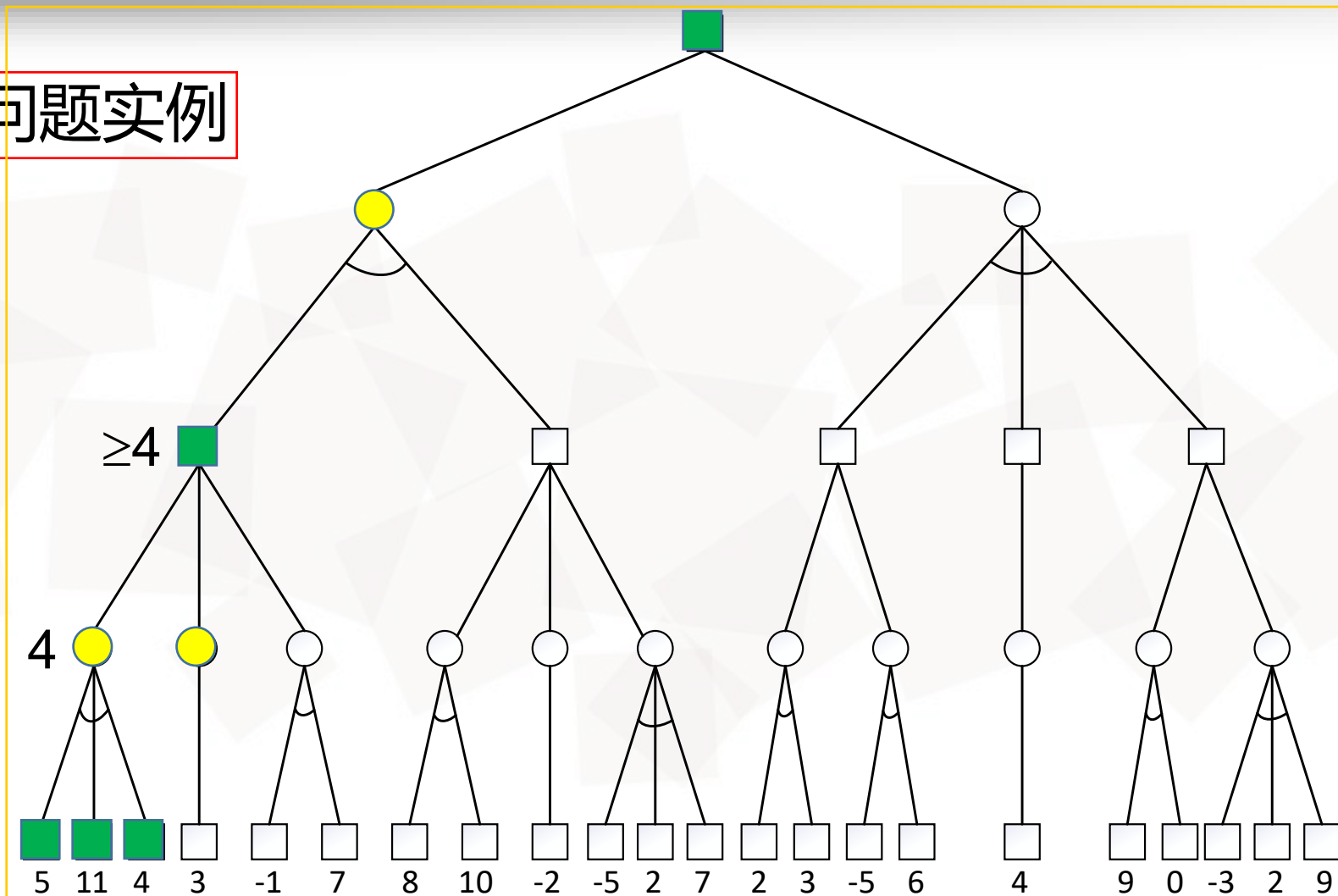
## $\alpha$ - $\beta$ 剪枝问题实例



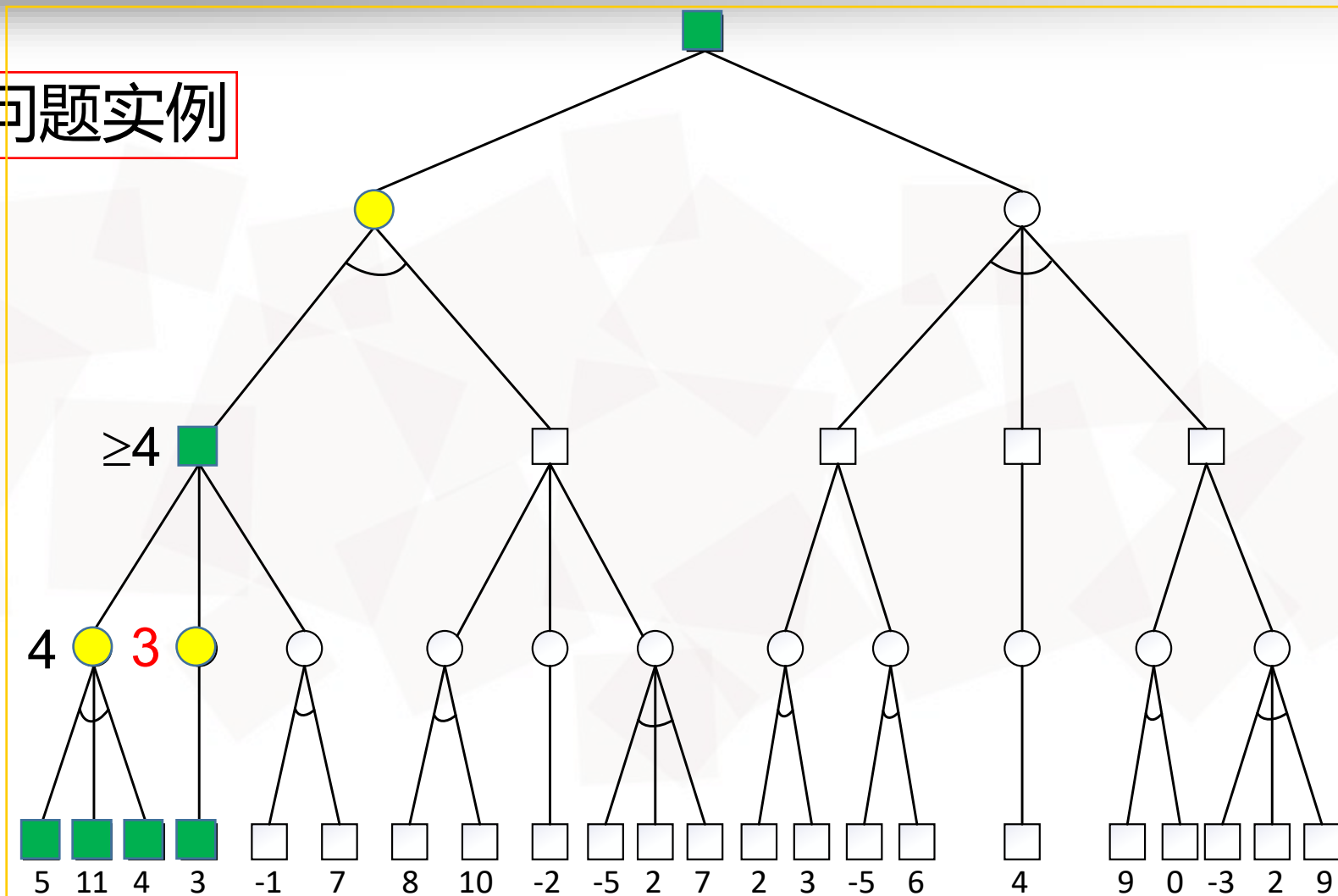
## $\alpha$ - $\beta$ 剪枝问题实例



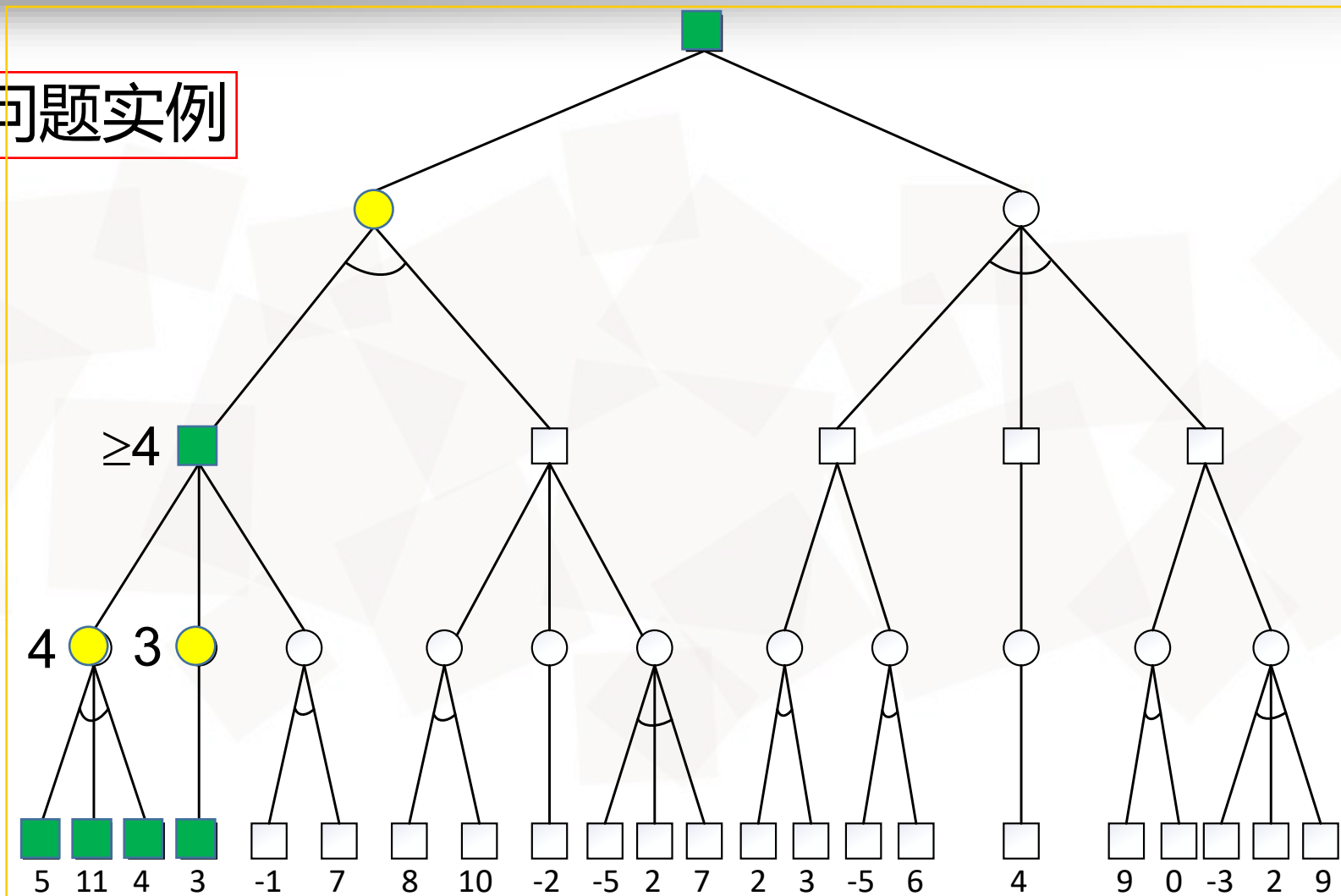
## $\alpha$ - $\beta$ 剪枝问题实例



## $\alpha$ - $\beta$ 剪枝问题实例

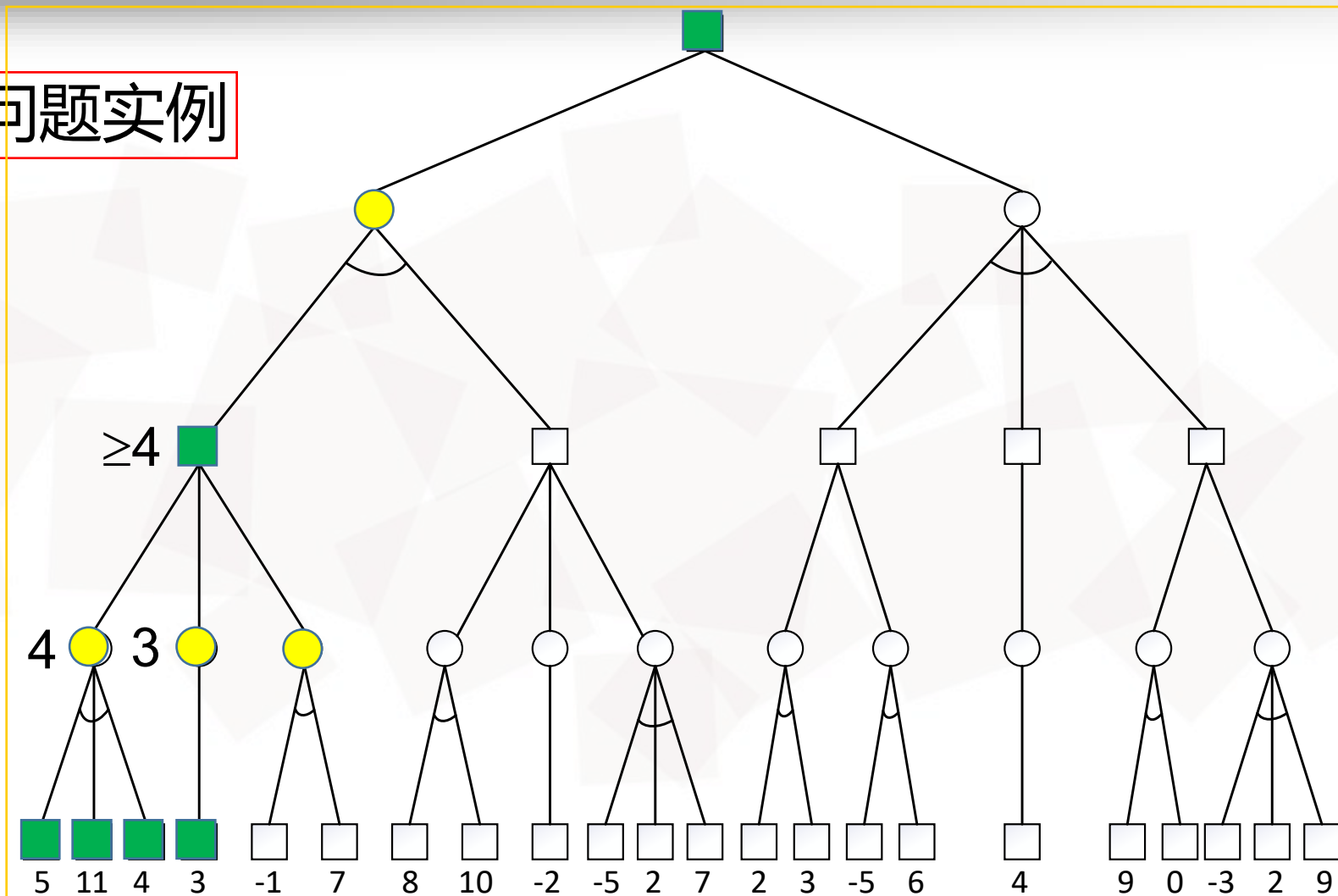


## $\alpha$ - $\beta$ 剪枝问题实例

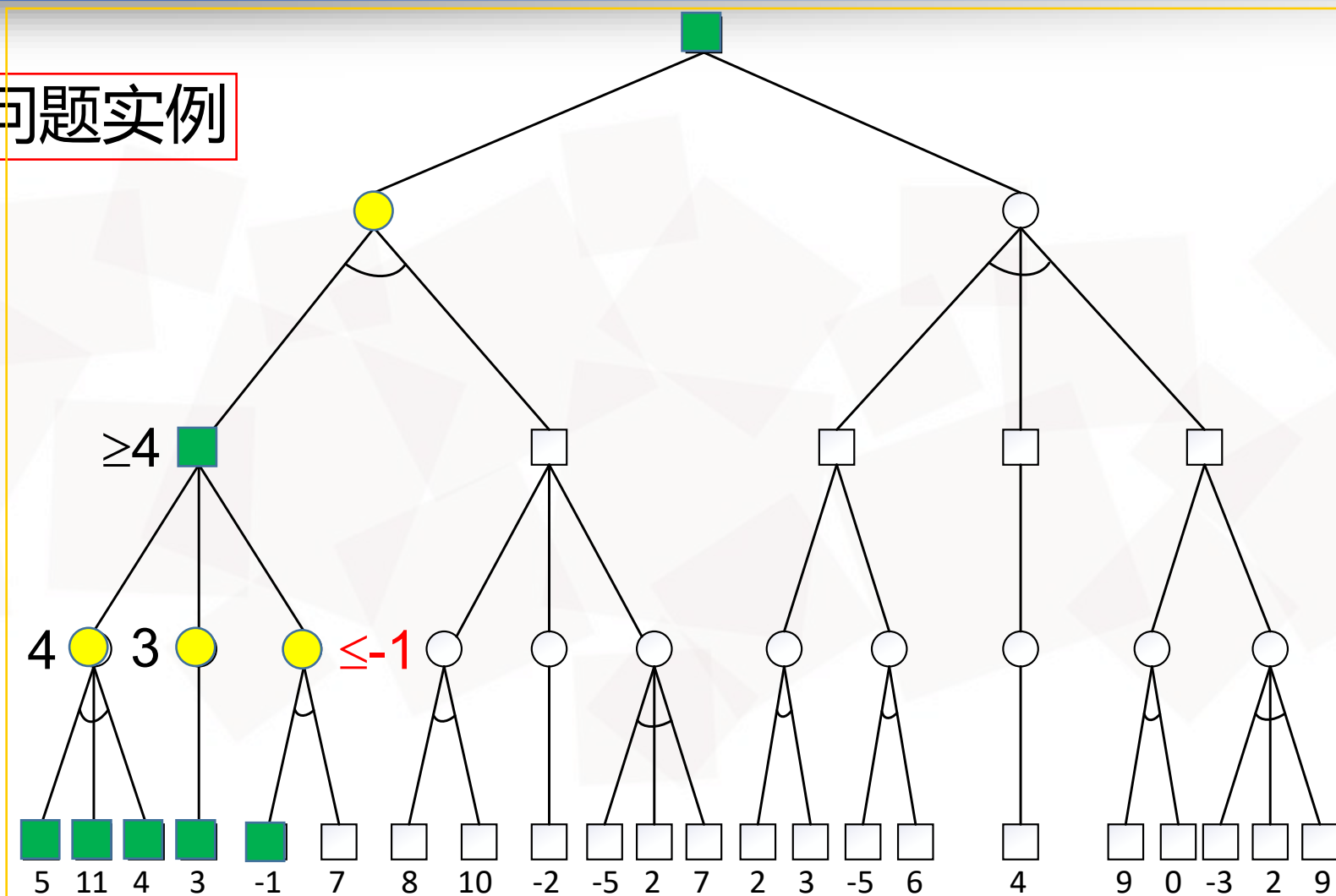




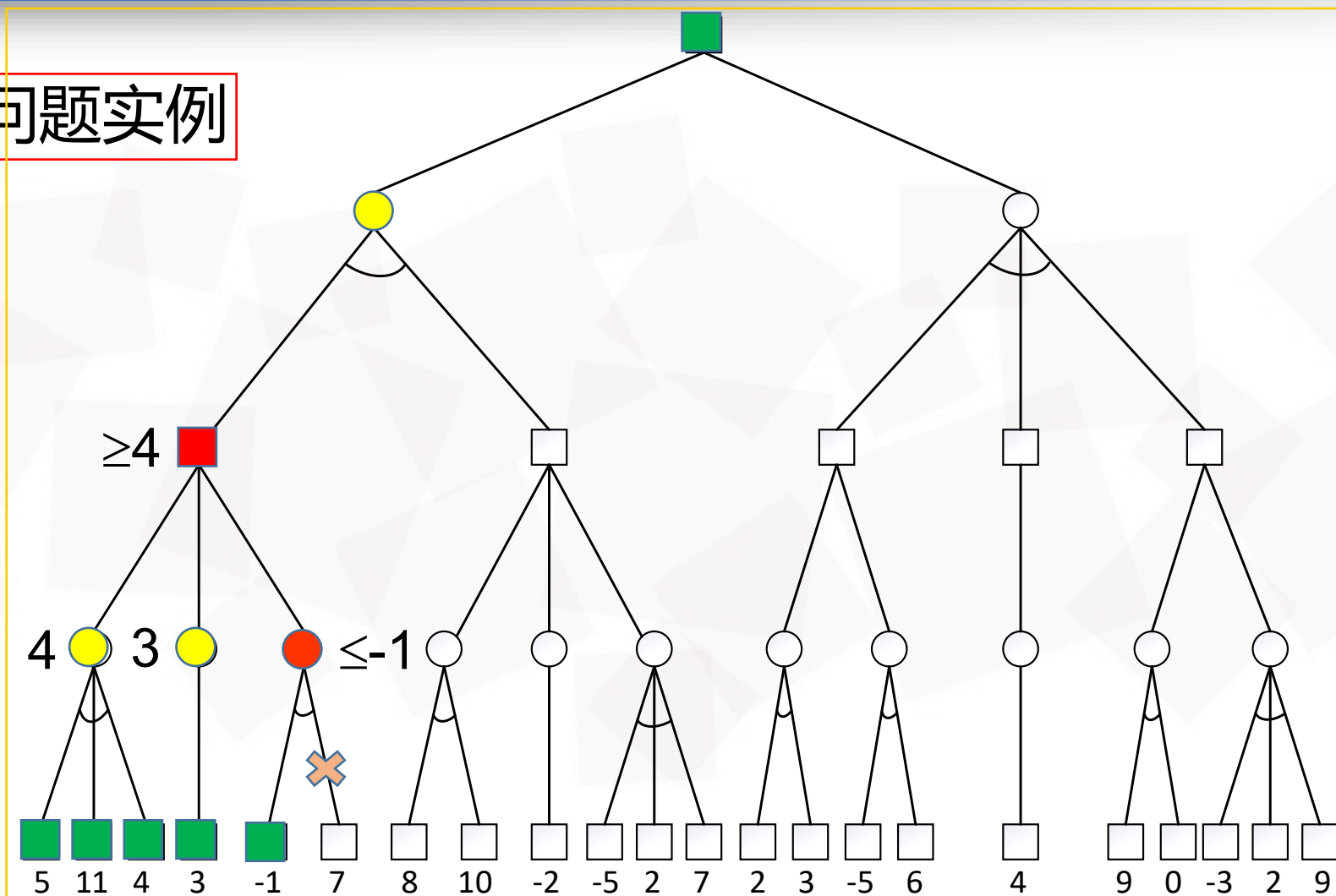
## $\alpha$ - $\beta$ 剪枝问题实例



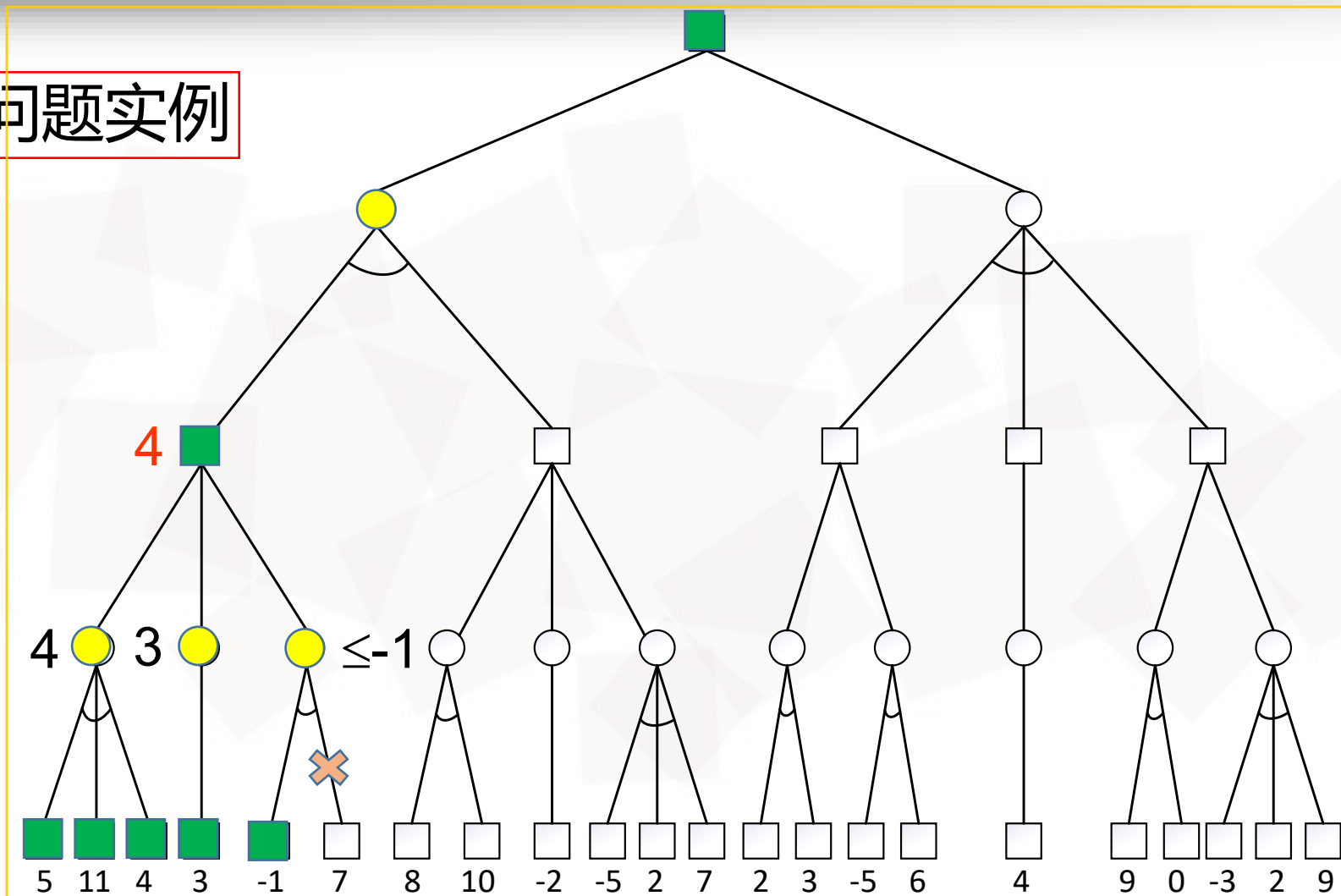
## $\alpha$ - $\beta$ 剪枝问题实例



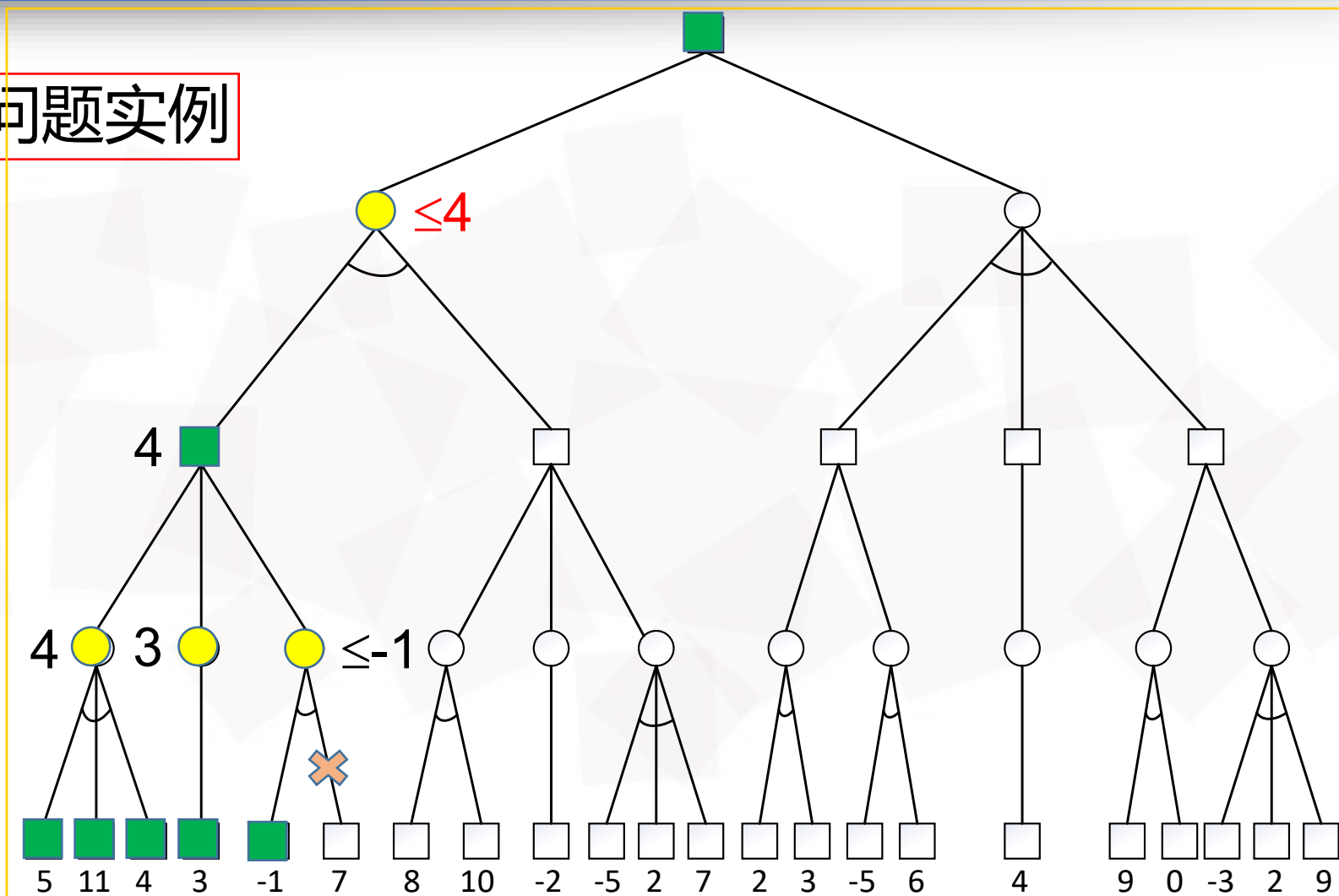
## $\alpha$ - $\beta$ 剪枝问题实例



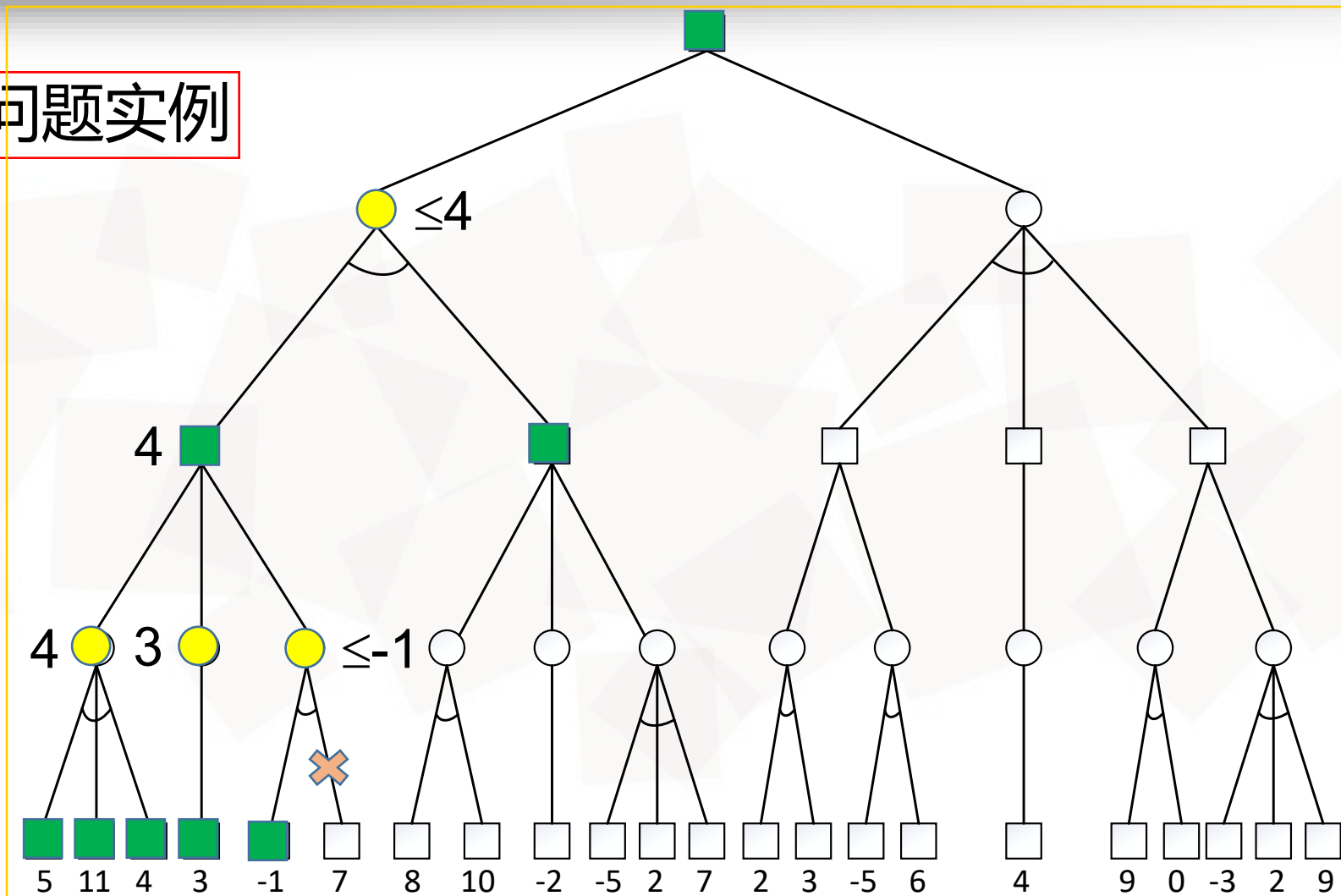
## $\alpha$ - $\beta$ 剪枝问题实例



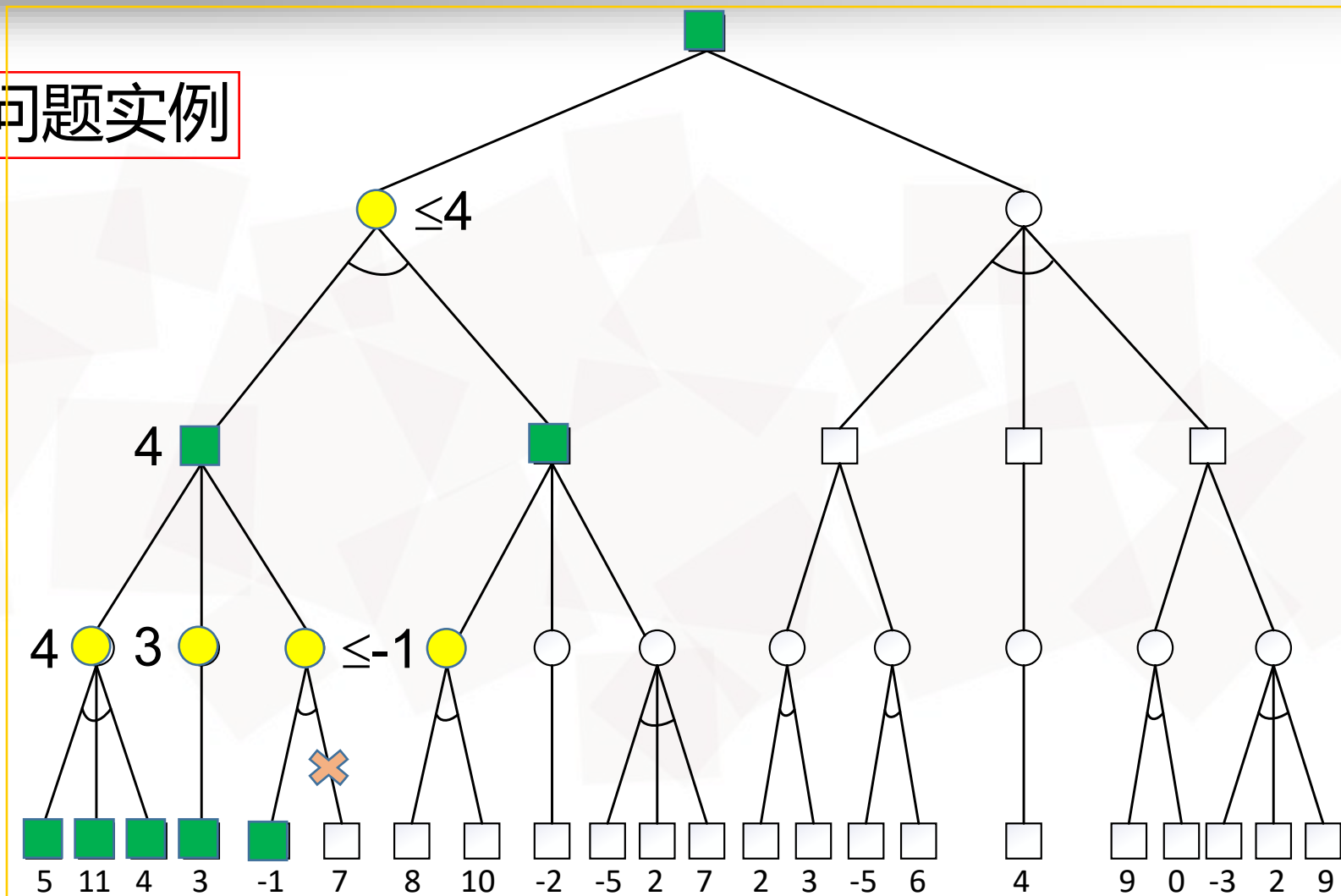
## $\alpha$ - $\beta$ 剪枝问题实例



## $\alpha$ - $\beta$ 剪枝问题实例



## $\alpha$ - $\beta$ 剪枝问题实例

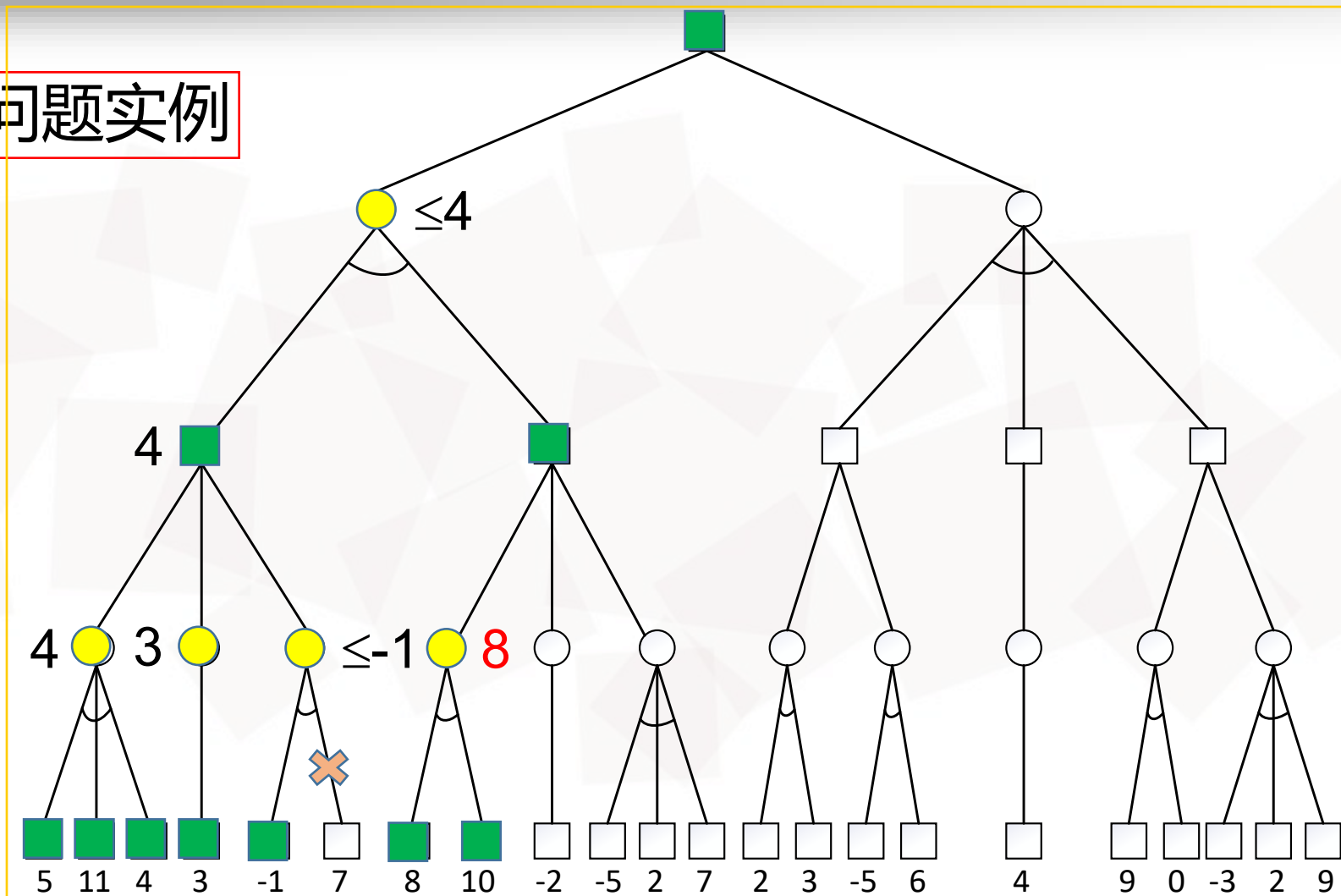


## $\alpha$ - $\beta$ 剪枝问题实例

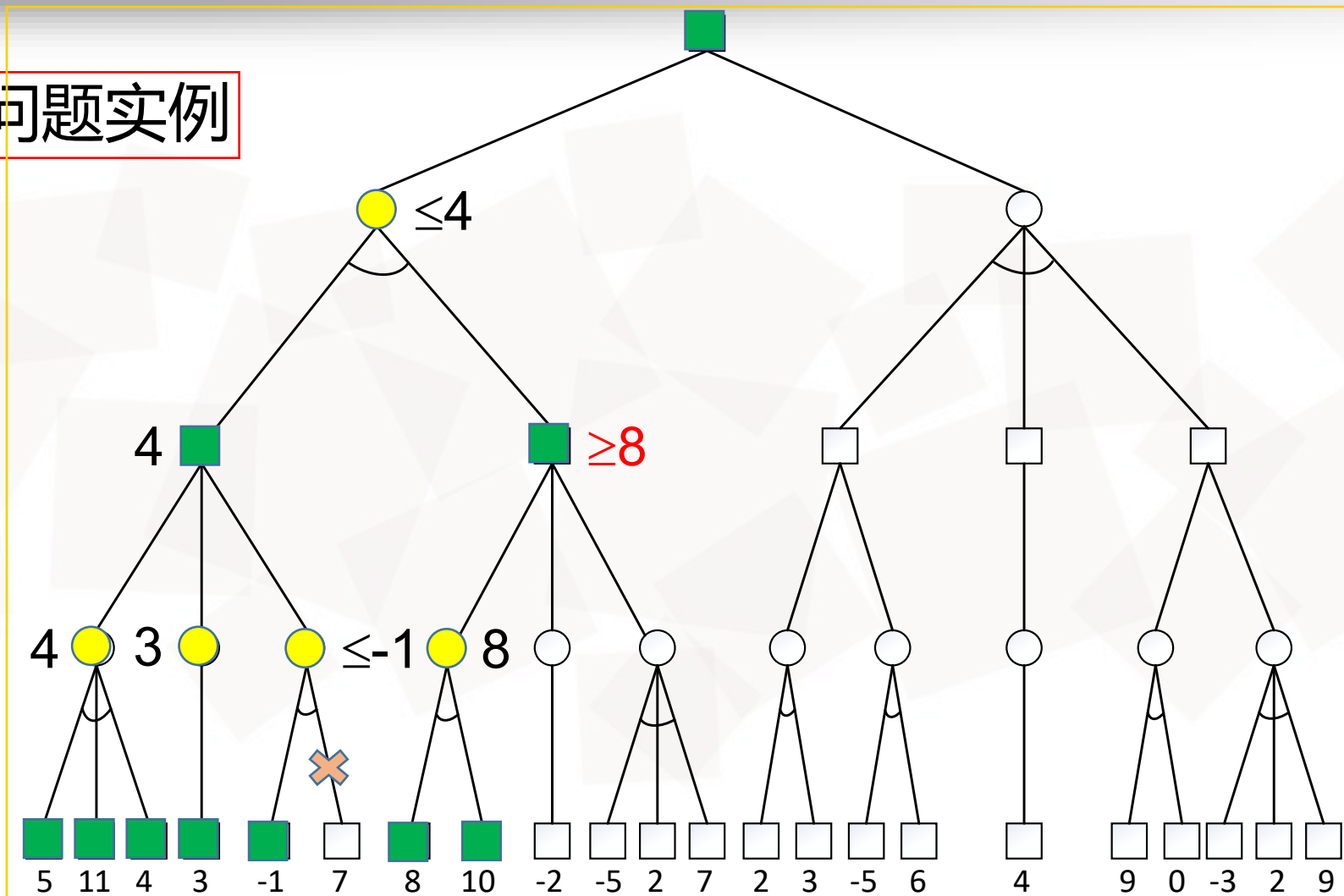




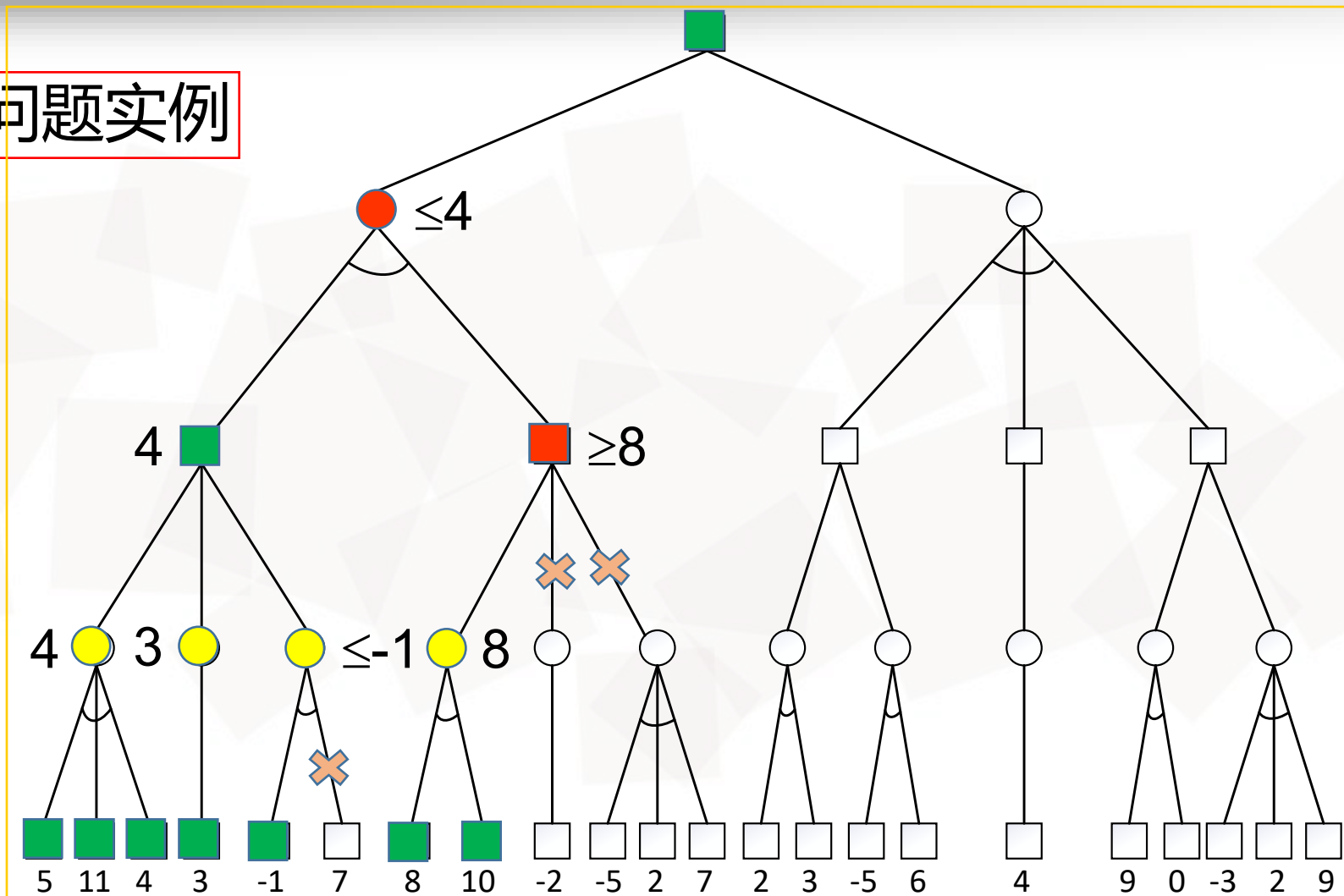
## $\alpha$ - $\beta$ 剪枝问题实例



## $\alpha$ - $\beta$ 剪枝问题实例



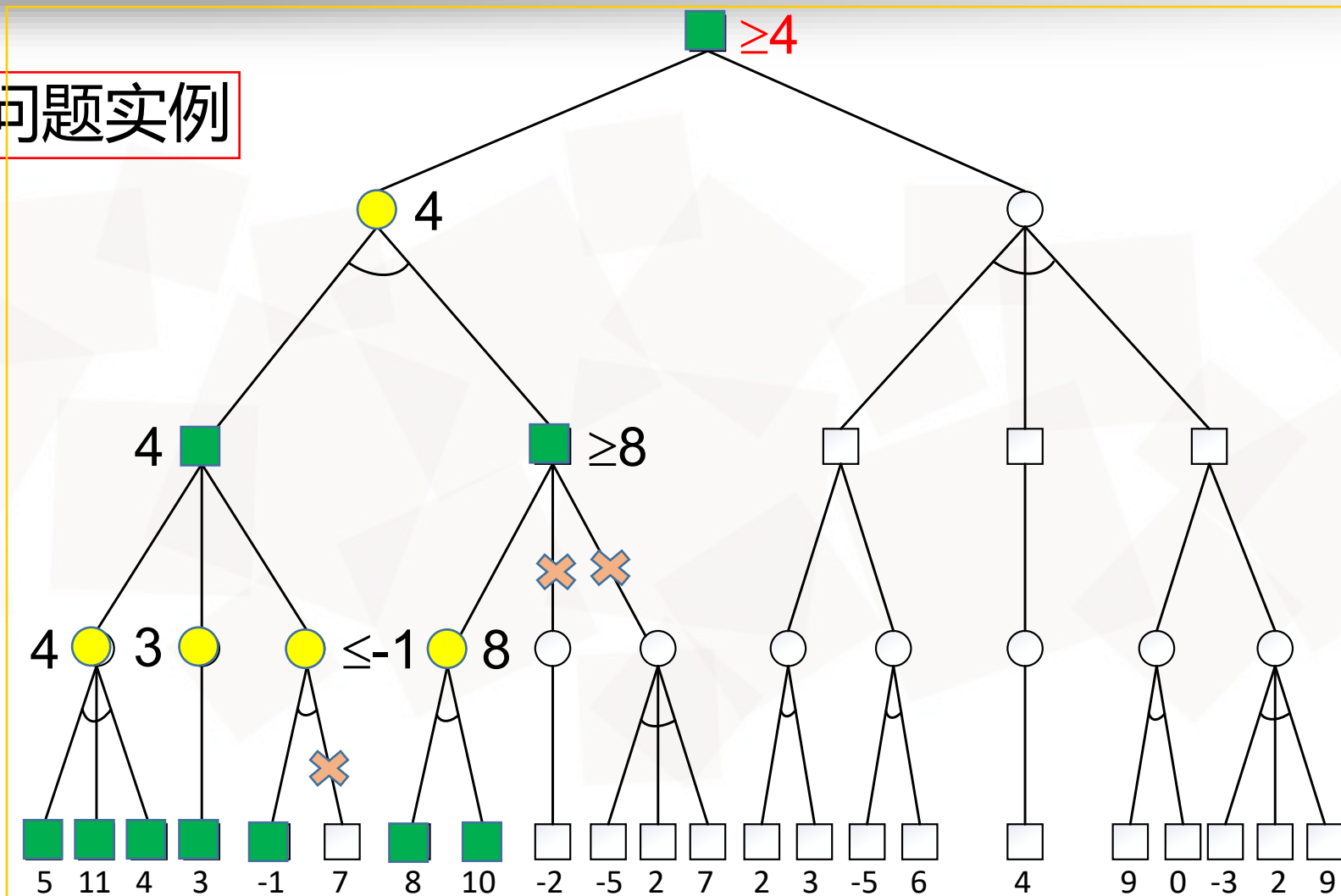
## $\alpha$ - $\beta$ 剪枝问题实例



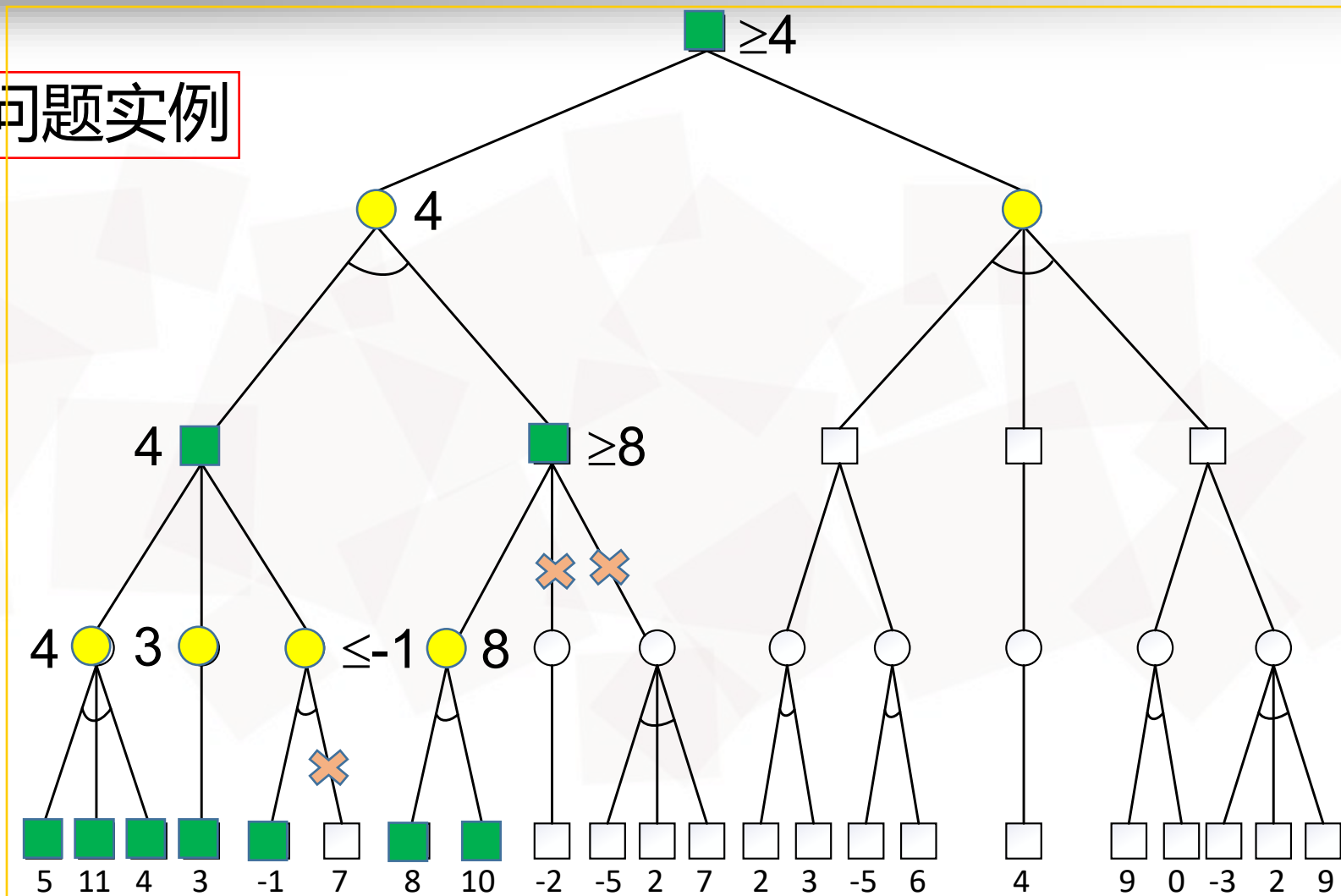
## $\alpha$ - $\beta$ 剪枝问题实例



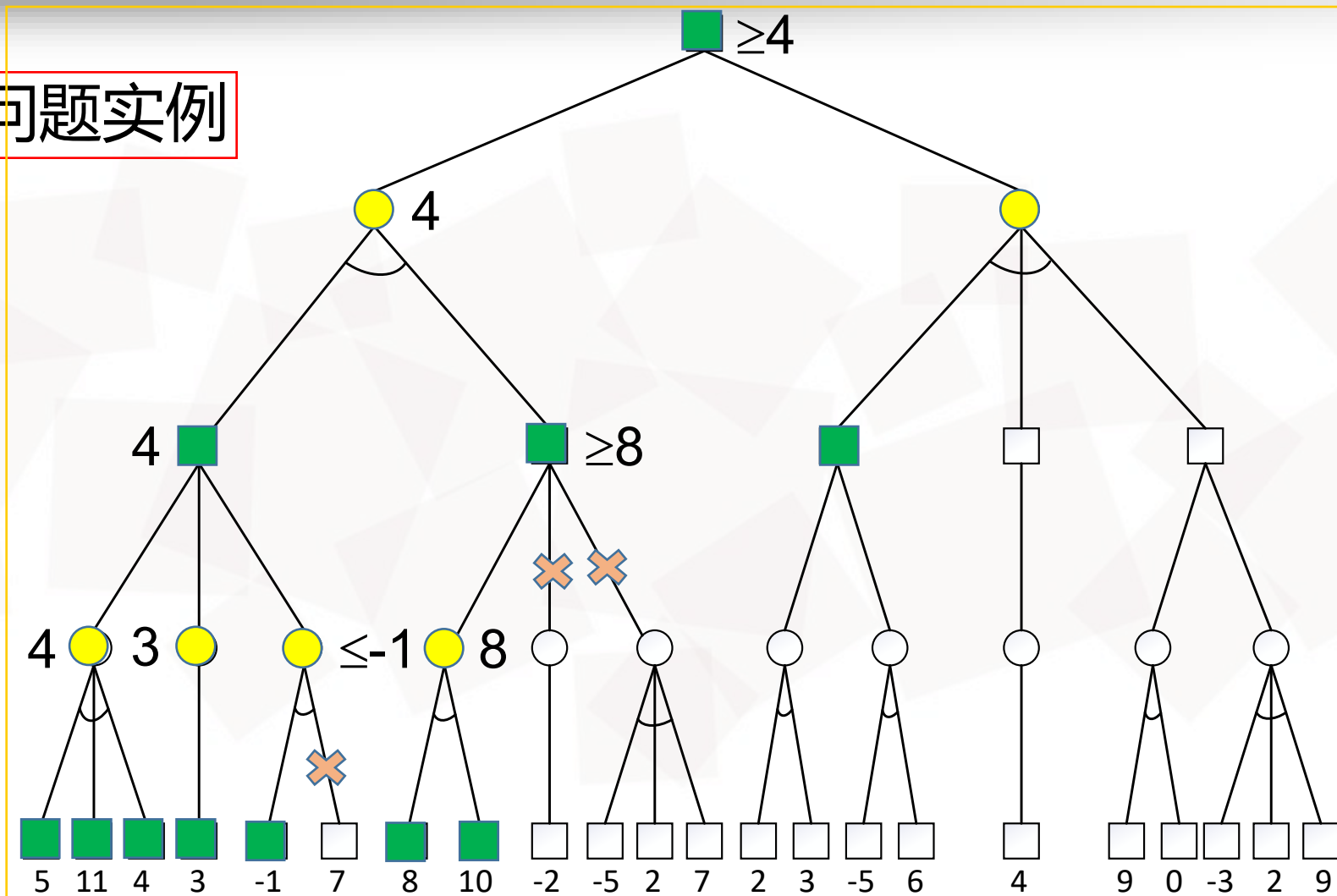
## $\alpha$ - $\beta$ 剪枝问题实例



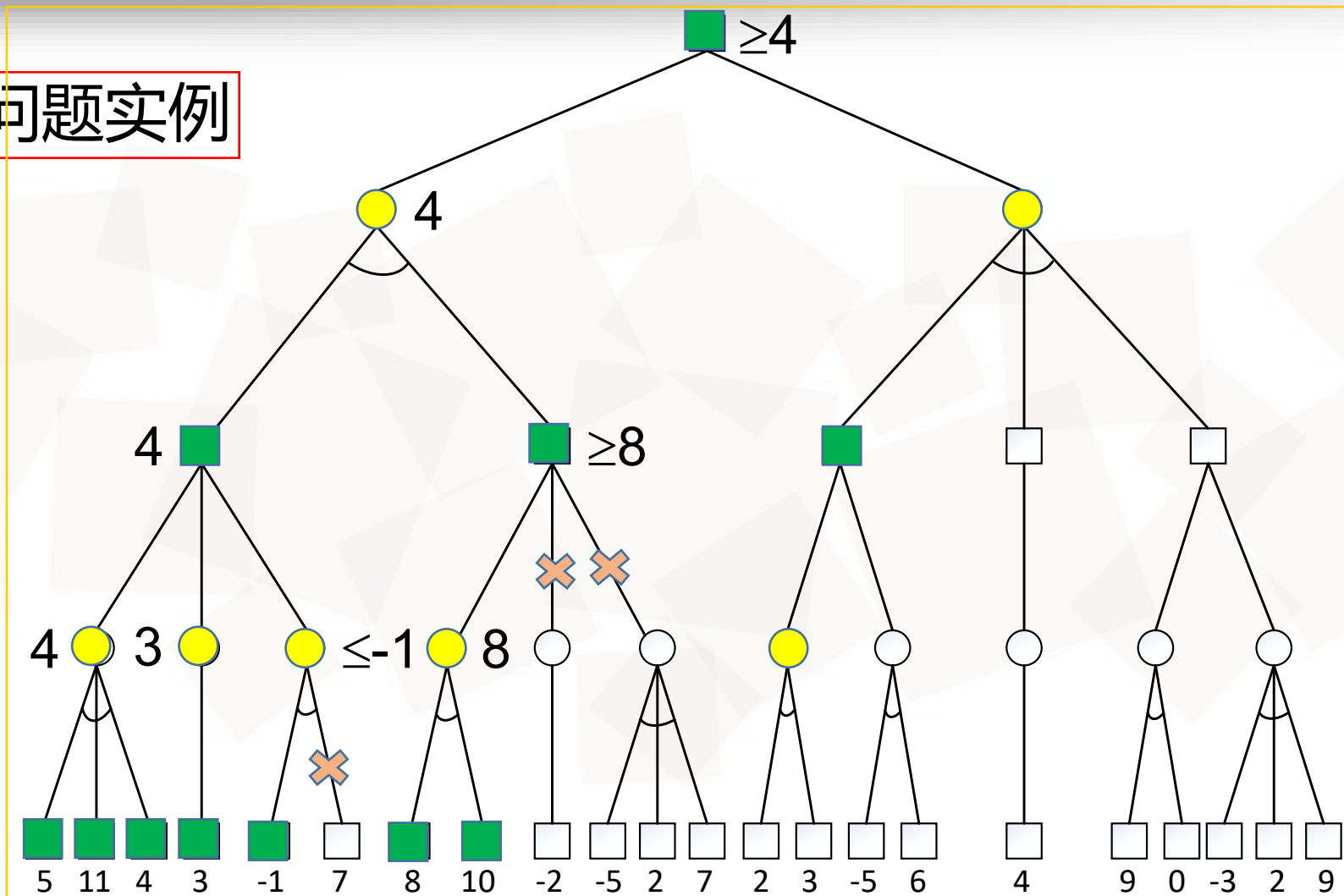
## $\alpha$ - $\beta$ 剪枝问题实例



## $\alpha$ - $\beta$ 剪枝问题实例

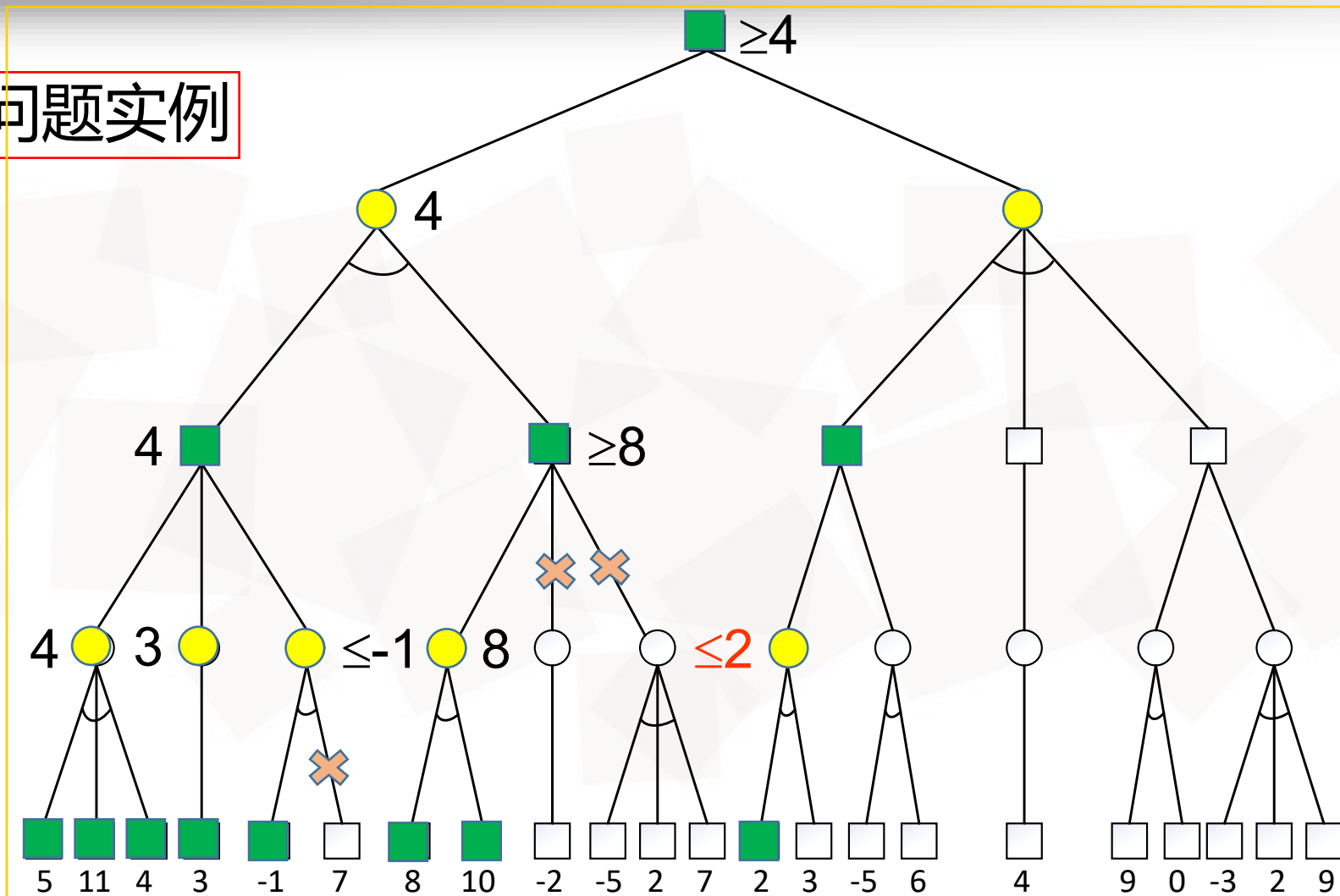


## $\alpha$ - $\beta$ 剪枝问题实例

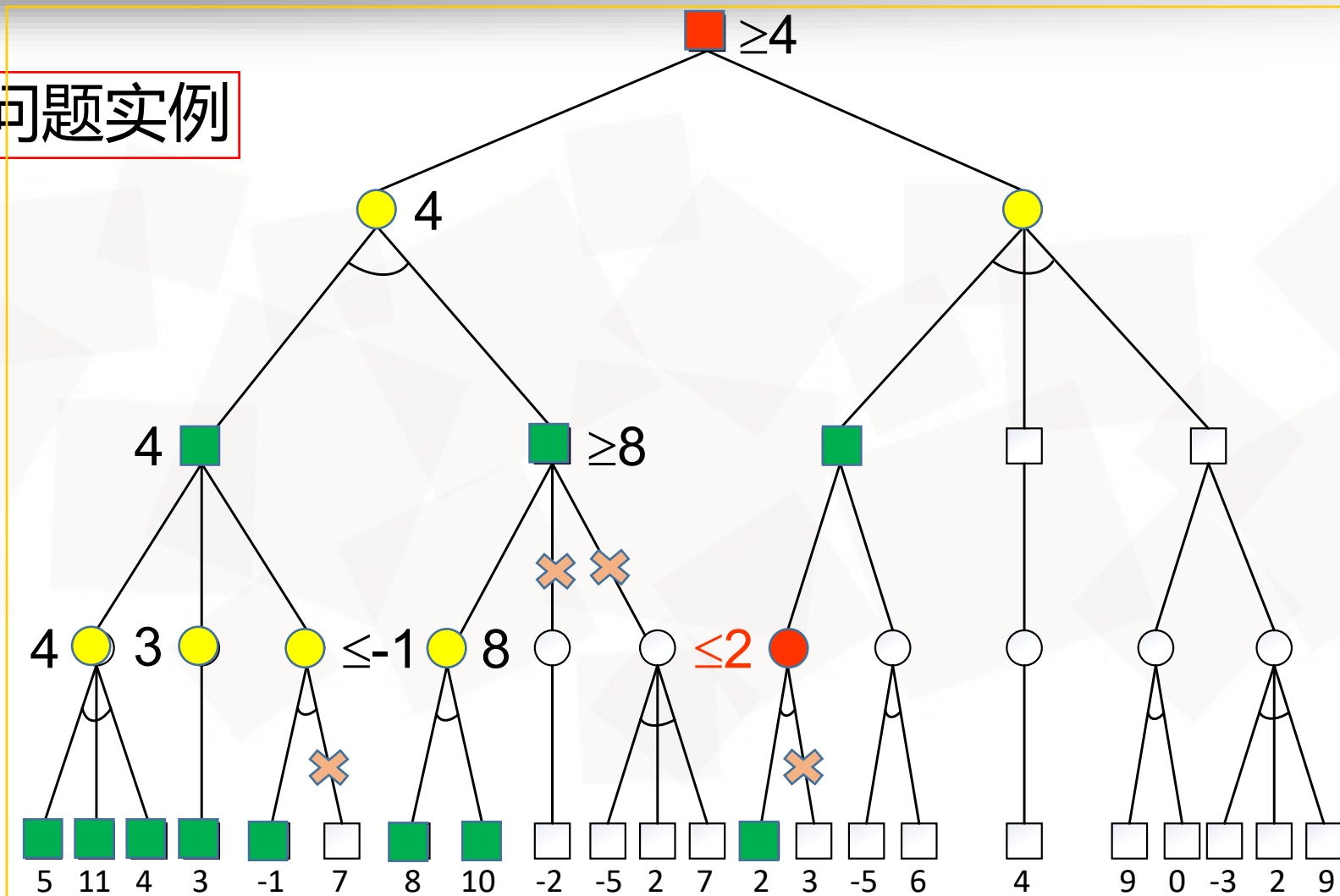




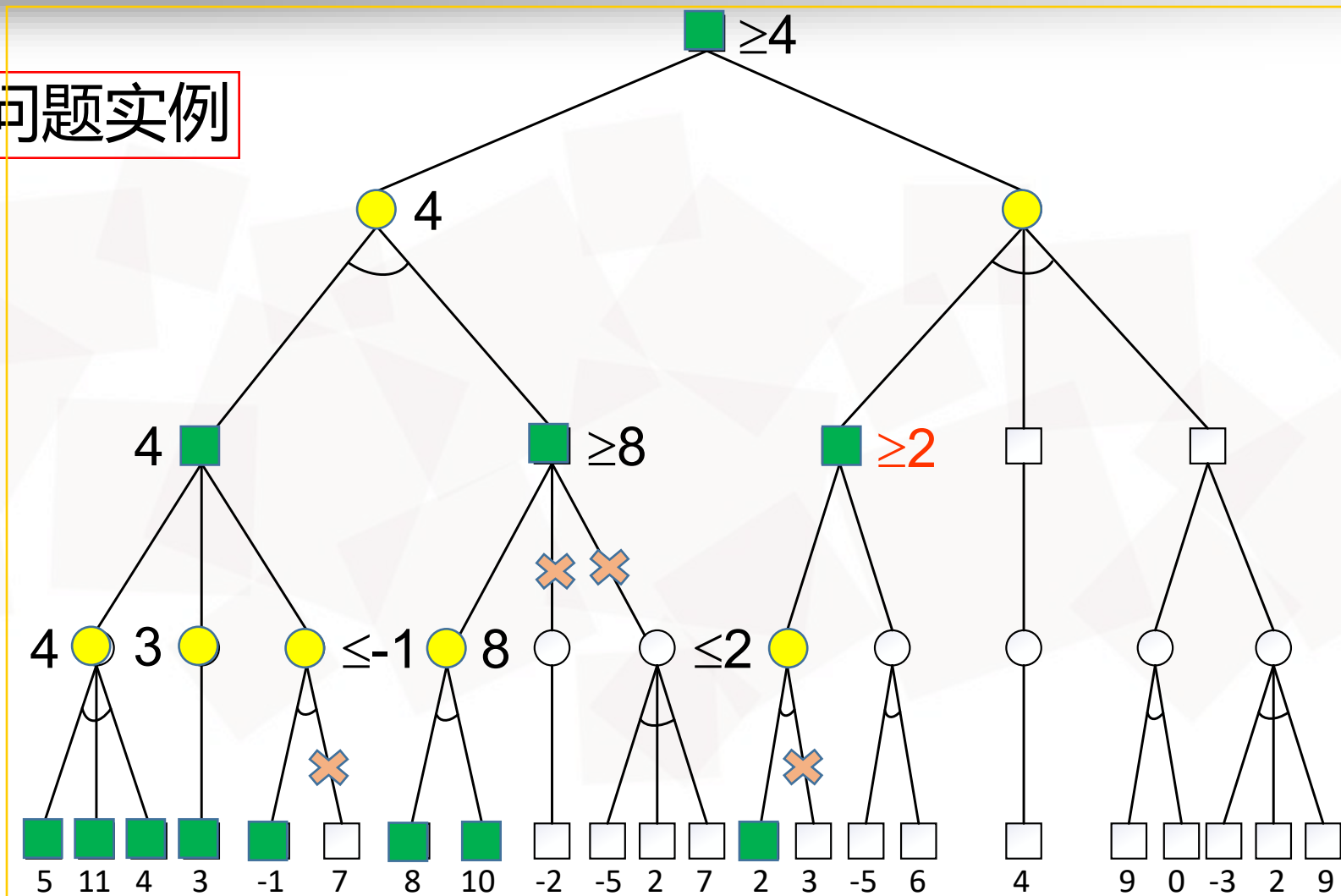
## $\alpha$ - $\beta$ 剪枝问题实例



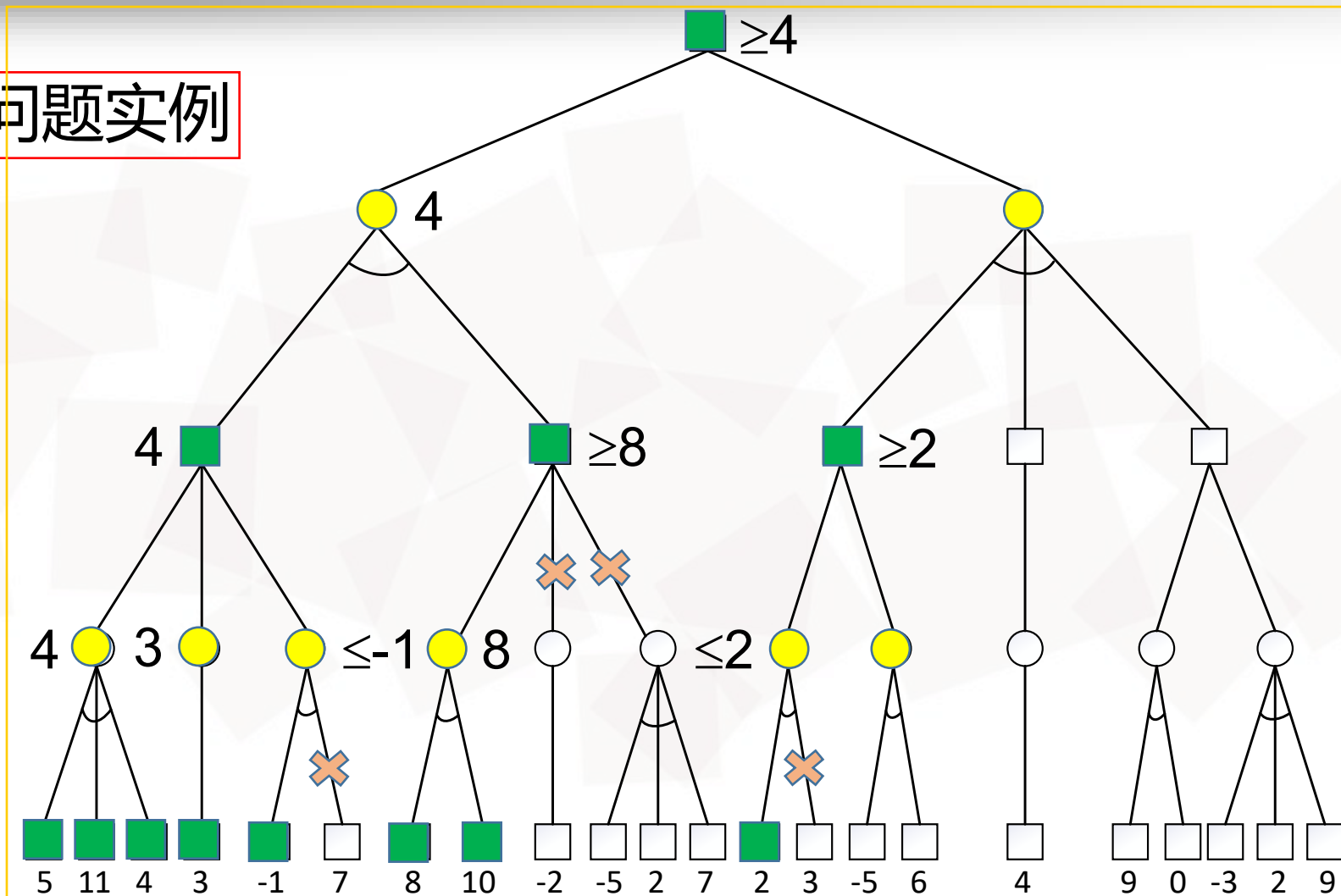
## $\alpha$ - $\beta$ 剪枝问题实例



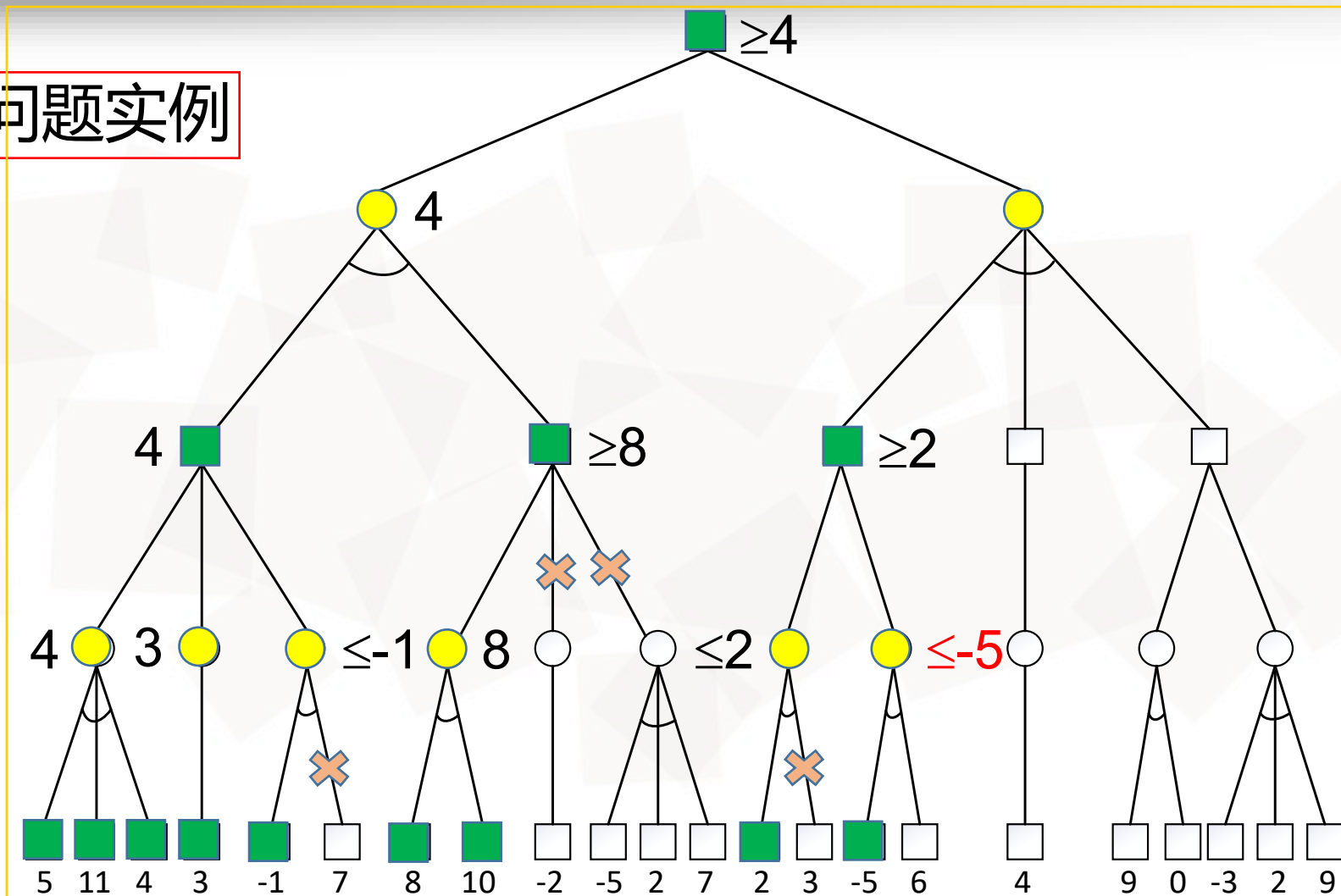
## $\alpha$ - $\beta$ 剪枝问题实例



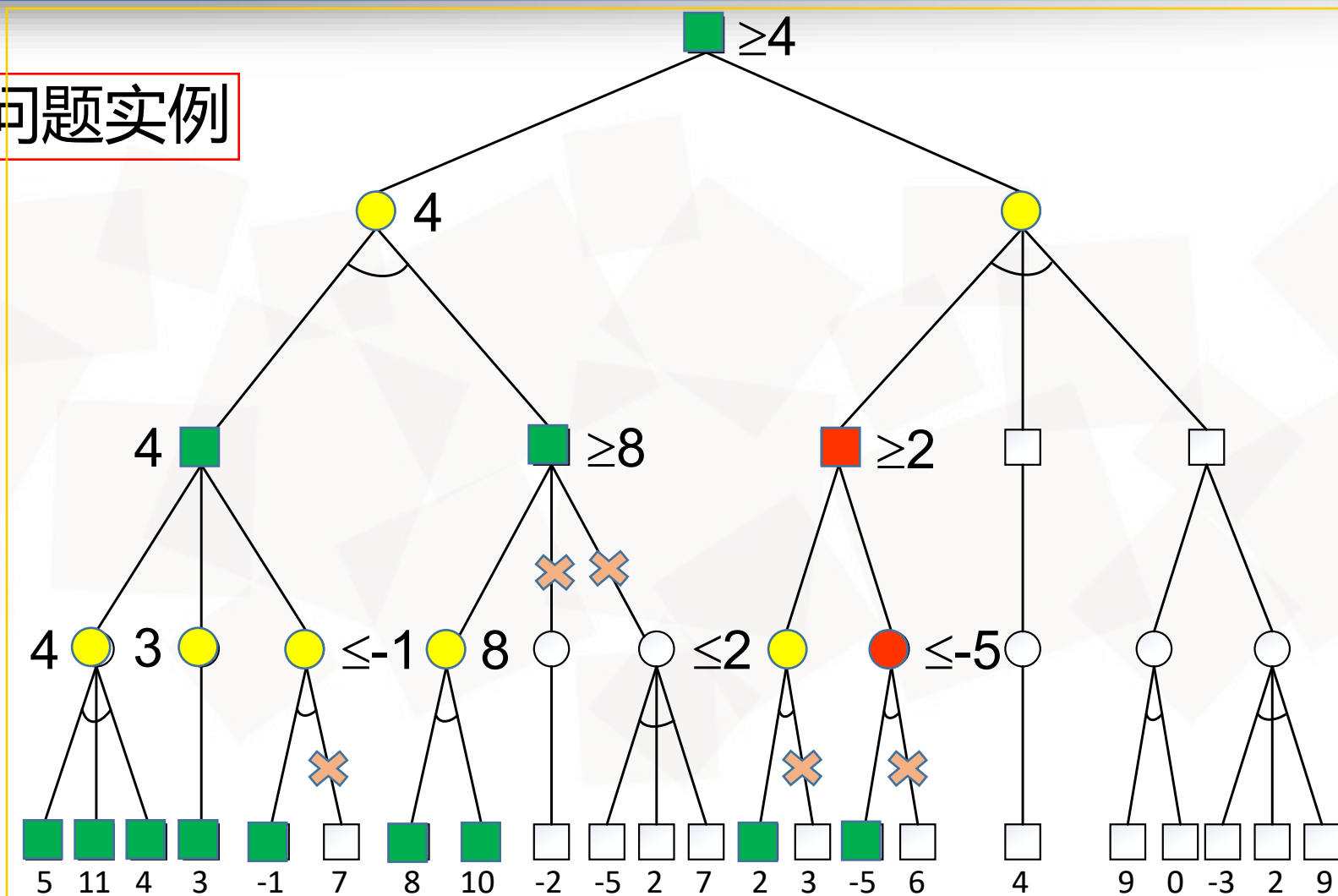
## $\alpha$ - $\beta$ 剪枝问题实例



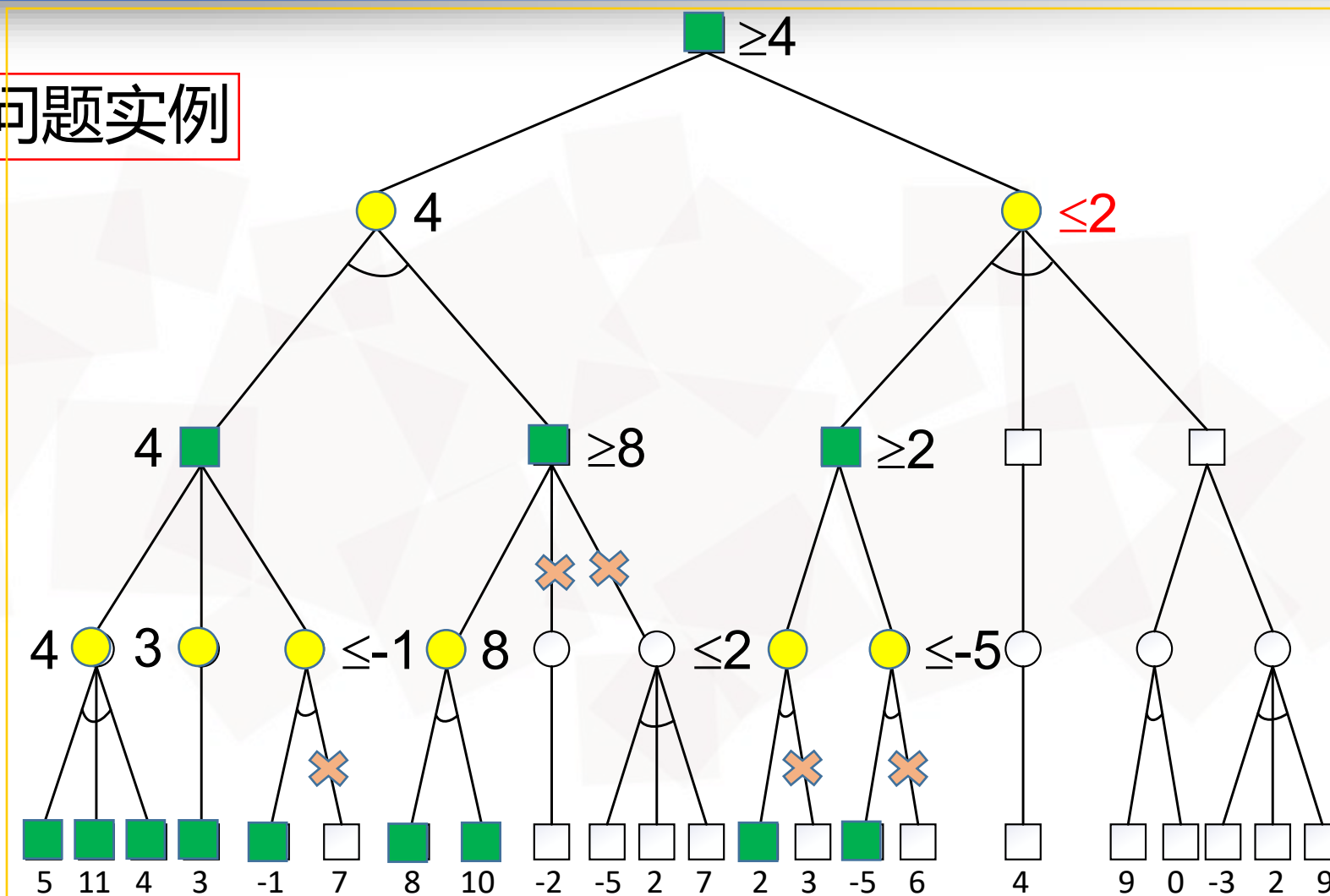
## $\alpha$ - $\beta$ 剪枝问题实例



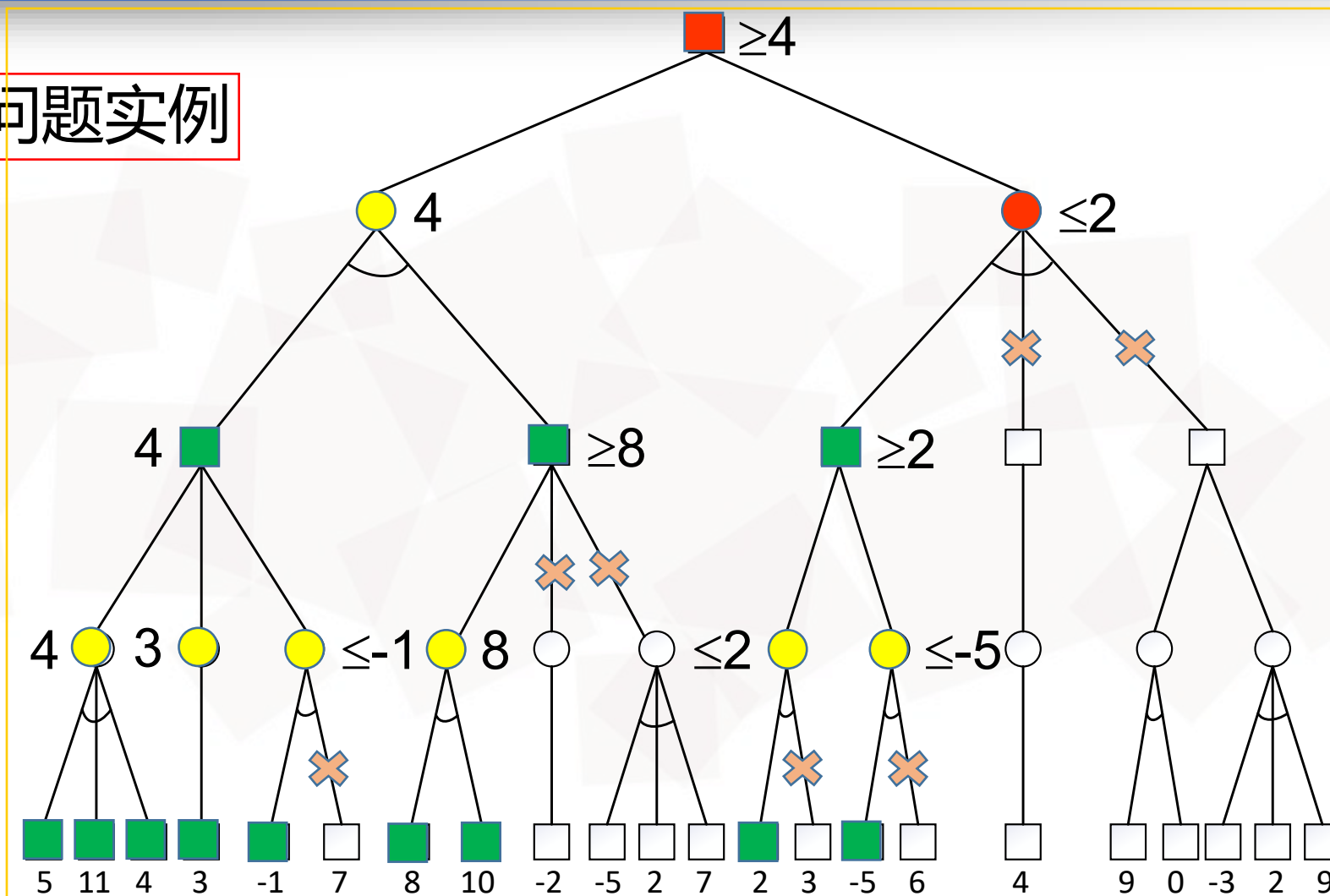
## $\alpha$ - $\beta$ 剪枝问题实例



## $\alpha$ - $\beta$ 剪枝问题实例

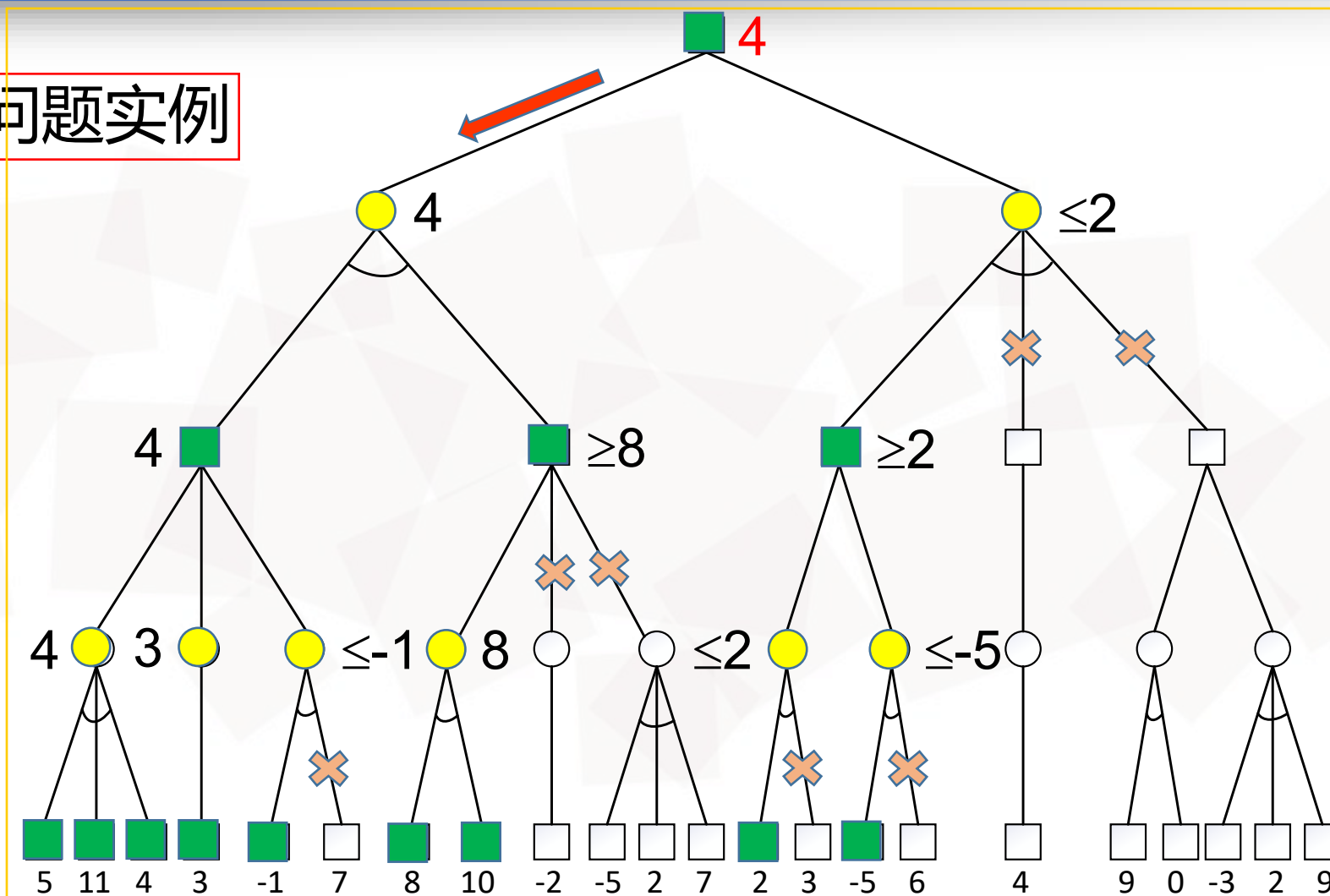


## $\alpha$ - $\beta$ 剪枝问题实例





## $\alpha$ - $\beta$ 剪枝问题实例

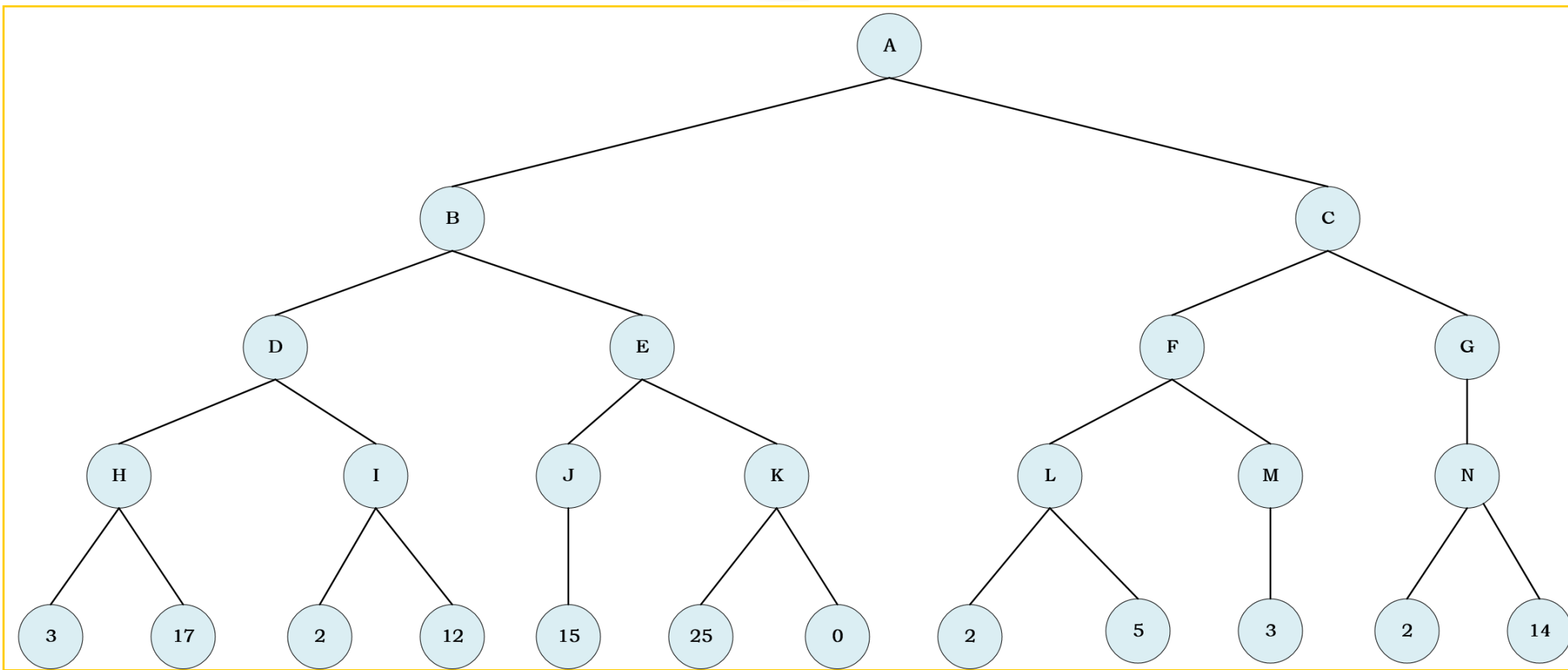


MAX

MIN

MAX

MIN



$\alpha$ - $\beta$ 剪枝可以将整个子树剪枝

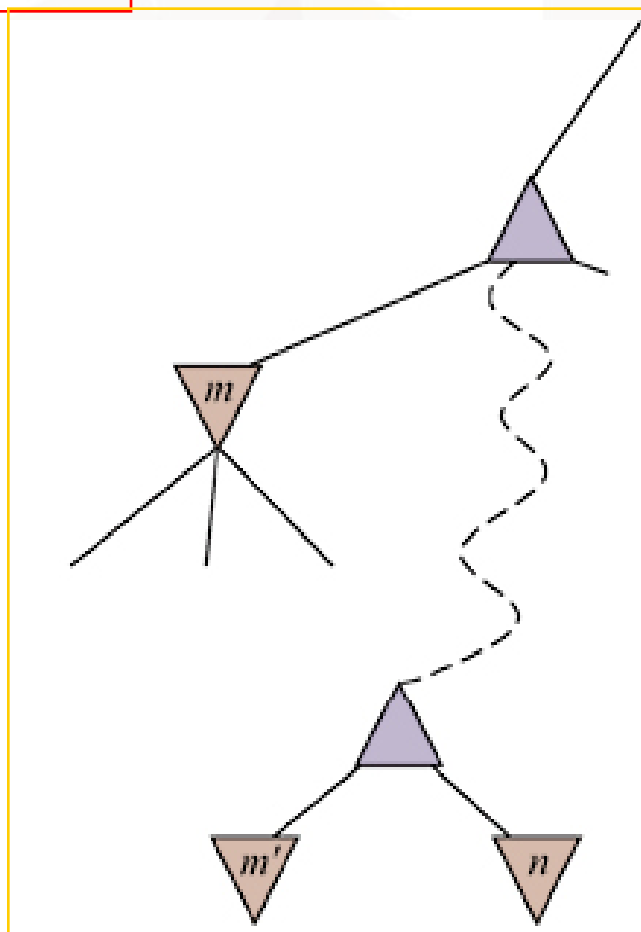
玩家

对手

⋮

玩家

对手





## $\alpha$ - $\beta$ 剪枝

- $\alpha$ : 到目前为止, 路径上发现的MAX的任一选择点中最佳 (即最大值) 选择的值。也就是说,  $\alpha$ 表示 “至少”
- $\beta$ : 到目前为止, 路径上发现的MIN的任一选择点中最佳 (即最小值) 选择的值。也就是说,  $\beta$ 表示 “至多”



## 搜索时间有限

- 启发式评价函数  $\text{Eval}(s, p)$  替代效用函数  $\text{Utility}(s, p)$ , 使非终止状态变为终止状态
- 截断测试替代终止测试, 可根据搜索深度和当前状态的任意属性自由决定何时终止搜索



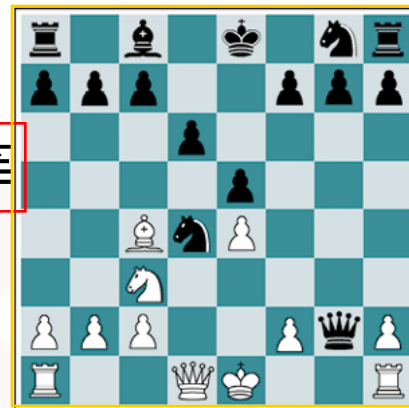
## 评价函数

- 启发式评价函数  $\text{Eval}(s, p)$ : 向参与者 $p$ 返回状态 $s$ 的期望效用的估计值
- 终止状态:  $\text{Eval}(s, p) = \text{Utility}(s, p)$
- 非终止状态:  $\text{Utility}(\text{loss}, p) \leq \text{Eval}(s, p) \leq \text{Utility}(\text{win}, p)$
- 评价函数的特点:
  - 计算时间不能太长
  - 应与实际的获胜机会密切相关

# 启发式 $\alpha$ - $\beta$ 树搜索

## 评价函数

- 特征：如白兵的数目、黑兵的数目、白后的数目、黑后的数目等
- 类别：同一类别中的状态对所有特征都具有相同值
  - 如包含所有“两兵对一兵”残局
- 任何给定类别都可能包含一些通往胜利的状态、通往平局的状态、通往失败的状态
  - 如“两兵对一兵”残局82%胜利(+1), 2%失败(0), 16%平局(1/2)
  - 期望值  $(0.82*1) + (0.02*0) + (0.16*1/2) = 0.90$





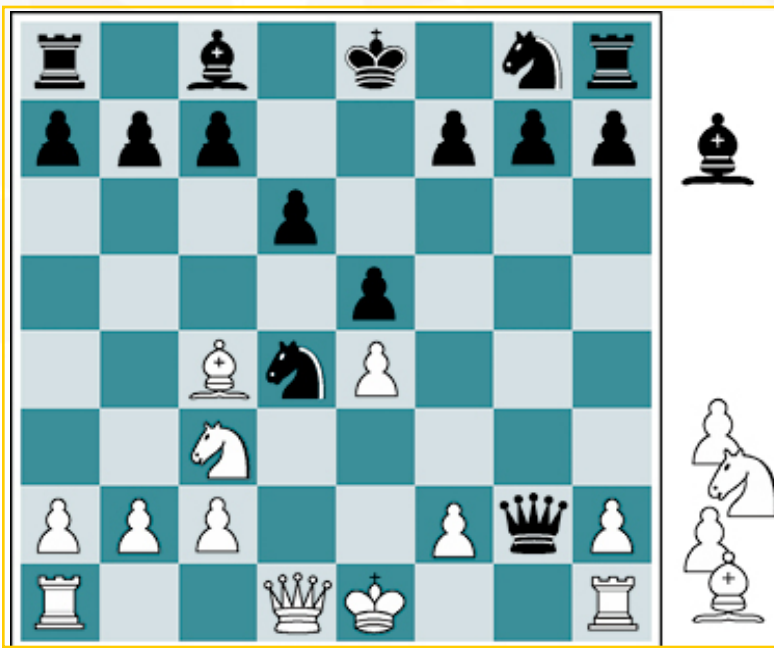
## 评价函数

- 计算每个特征的数值贡献，特征值简单地相加得到局面评估值
  - 国际象棋各个棋子的子力价值：兵1分、马或象3分、车5分、后9分
- 加权线性函数:  $\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$ 
  - $f_i(s)$  表示局面特征
  - $w_i$  表示其权重，表明该特征的重要性，需归一化
- 线性假设：每个特征的贡献独立于其它特征的值

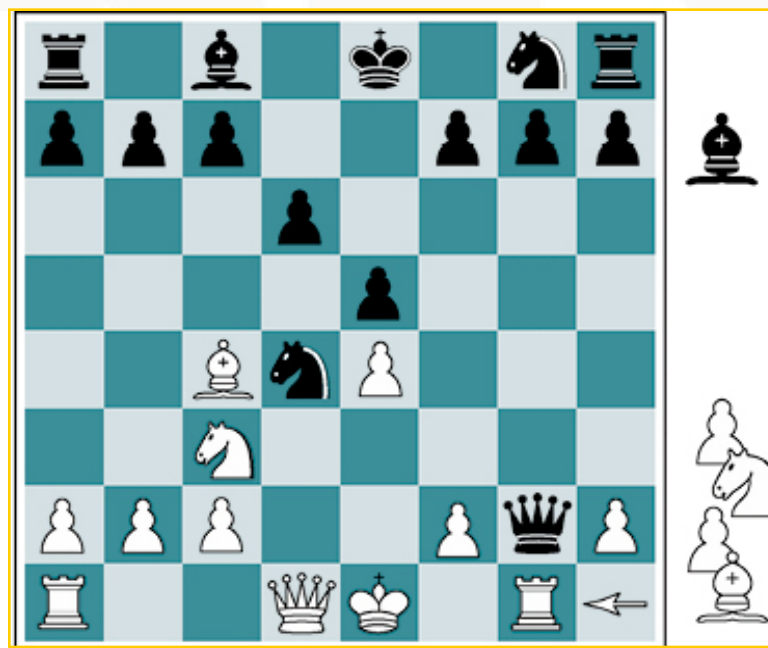


- 近似值可能导致误差

后往往是棋局中制胜的决定性力量，少掉一个后往往意味着**棋局告负**，此时失去后的一方通常会投子认输。



(a) 轮到白棋移动



(b) 轮到白棋移动

# 启发式 $\alpha$ - $\beta$ 树搜索

## 截断搜索

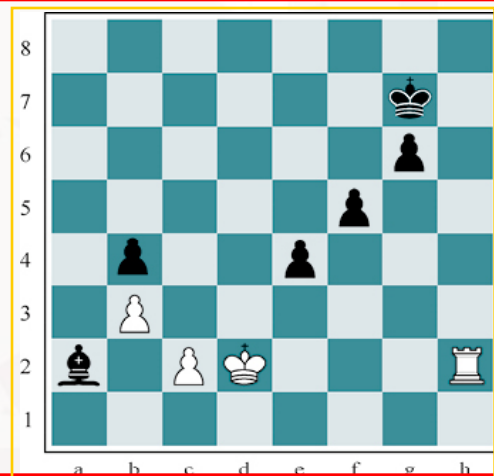
- If game.Is-Cutoff(state, depth) Then Return **game.Eval**(state, player), null

- 截断条件:

- 终止状态

- 大于指定深度d

- 迭代加深至时间耗尽



黑方移动后，黑象注定难逃厄运。但是黑方可以用兵来阻挡白方的王，引诱王去吃掉兵。这会将不可避免的象的损失推到视野之外，因此，搜索算法将牺牲兵的这一步看作“好招”

- 视野效应：程序面临一个给我方造成严重损失、基本无法避免的对方移动，可以使用拖延战术暂时避开
- 单步延伸：截断后允许算法继续沿着明显更好的搜索方向延伸



## 前向剪枝

- 剪掉看上去很糟糕但也可能实际很好的移动，以出错风险增大的代价节省了计算时间
- 束搜索，每层只考虑“一束”  $n$  个最佳移动
- ProbCut概率截断算法：剪除有可能位于 $\alpha$ - $\beta$ 窗口之外的节点
- 后期移动缩减算法：假设移动顺序已经调整好，后期出现的移动不太可能是好的移动，减少搜索这些移动的深度



## 搜索和查表

- 开局：复制人类专家关于如何打好每个开局的最佳建议并将其输入表中供计算机查表
- 中间局面：10-15步后，达到少见局面，程序切换到搜索
- 残局：游戏接近结束时，局面数量变少，更容易查表



## 蒙特卡罗树搜索

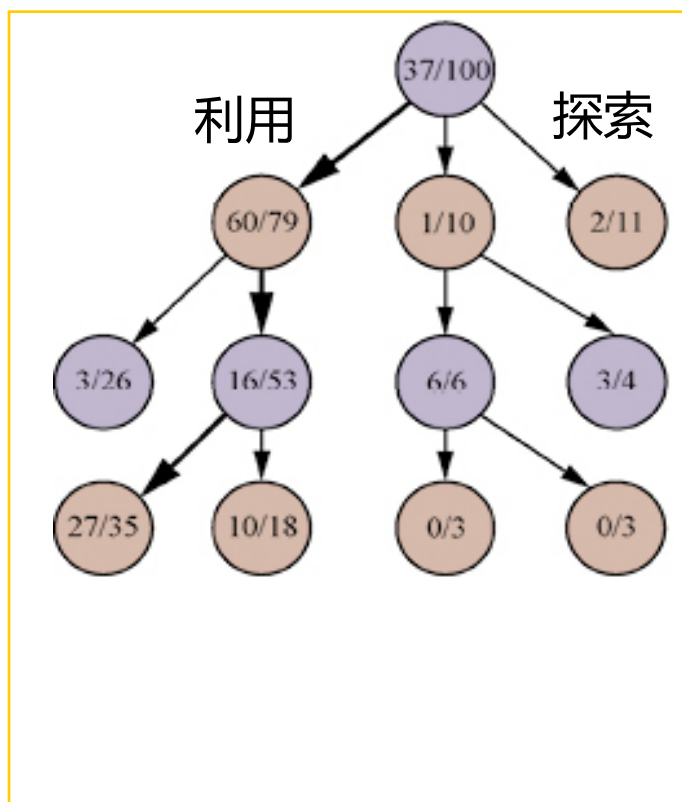
- **选择**: 从搜索树的根节点开始, 选择一个移动 (选择策略), 到达一个后继节点, 重复该过程, 沿着树向下移动到叶节点
- **扩展**: 为所选节点生成一个新的子节点 (0/0) 的方式增长搜索树
- **模拟**: 从新生成的子节点开始执行一次模拟 (模拟策略), 两个参与者选择移动, 但不记录在搜索树中
- **反向传播**: 使用模拟结果自底向上更新搜索树

白棋

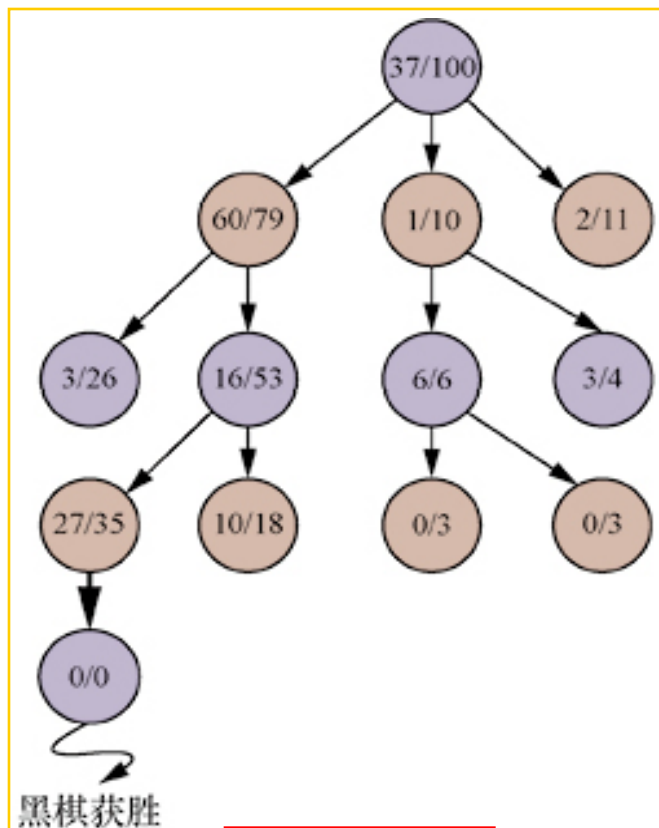
黑棋

白棋

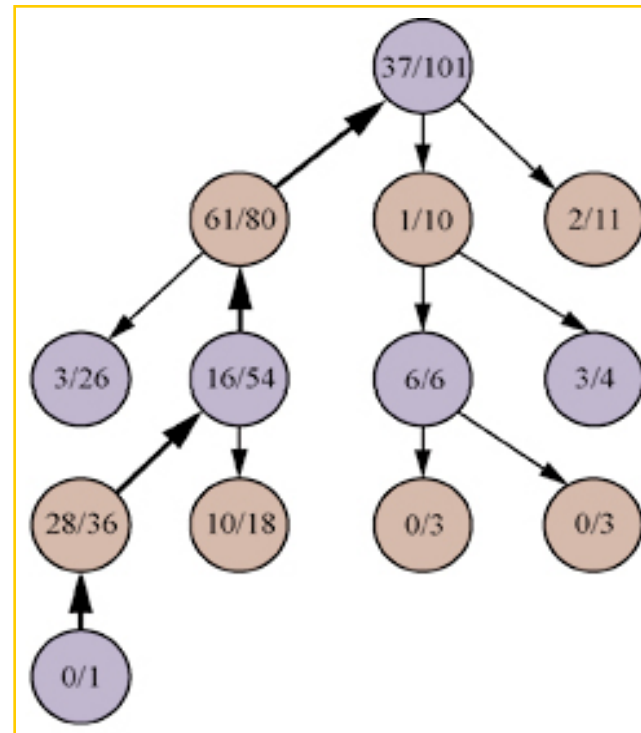
黑棋



(a) 选择



(b) 扩展和模拟



(c) 反向传播



## 蒙特卡罗树搜索算法

**function** MONTE-CARLO-TREE-SEARCH(*state*) **returns** 一个动作

*tree*  $\leftarrow$  NODE(*state*)

**while** IS-TIME-REMAINING() **do**

*leaf*  $\leftarrow$  SELECT(*tree*)

*child*  $\leftarrow$  EXPAND(*leaf*)

*result*  $\leftarrow$  SIMULATE(*child*)

  BACK-PROPAGATE(*result*, *child*)

**return** ACTIONS(*state*)中指向模拟次数最多的节点的移动

- 获胜65/100优于获胜2/3，后者有很多不确定性





## 优点

- 对于分支因子很高或者难以定义一个好的评价函数的游戏（围棋），蒙特卡罗搜索优于 $\alpha$ - $\beta$ 搜索
- 蒙特卡罗依赖于多次模拟的聚合，因此不容易受到单次错误的影响
- 二者结合，提前终止模拟
- 可以应用于没有任何经验定义评价函数的全新博弈，选择和模拟可利用人工经验或者探索过程得到的经验

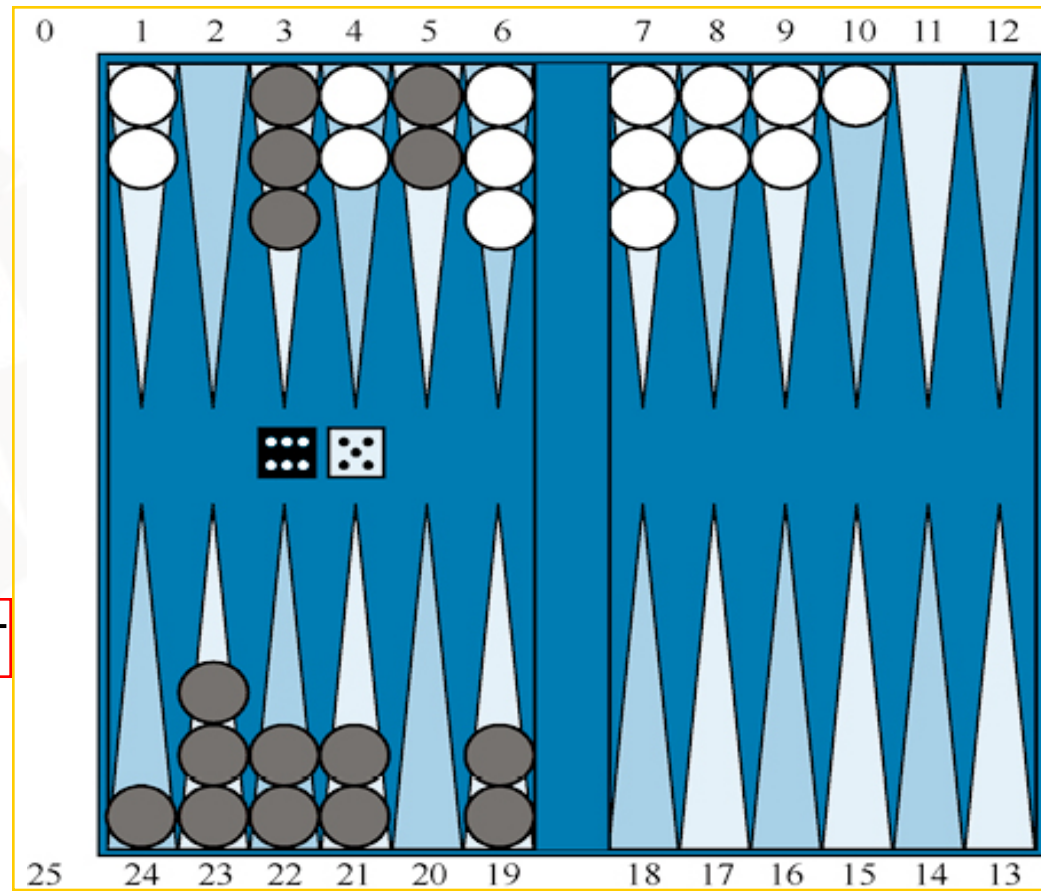
## 缺点

- 当双方正负明显时，仍需要模拟多步来验证获胜方



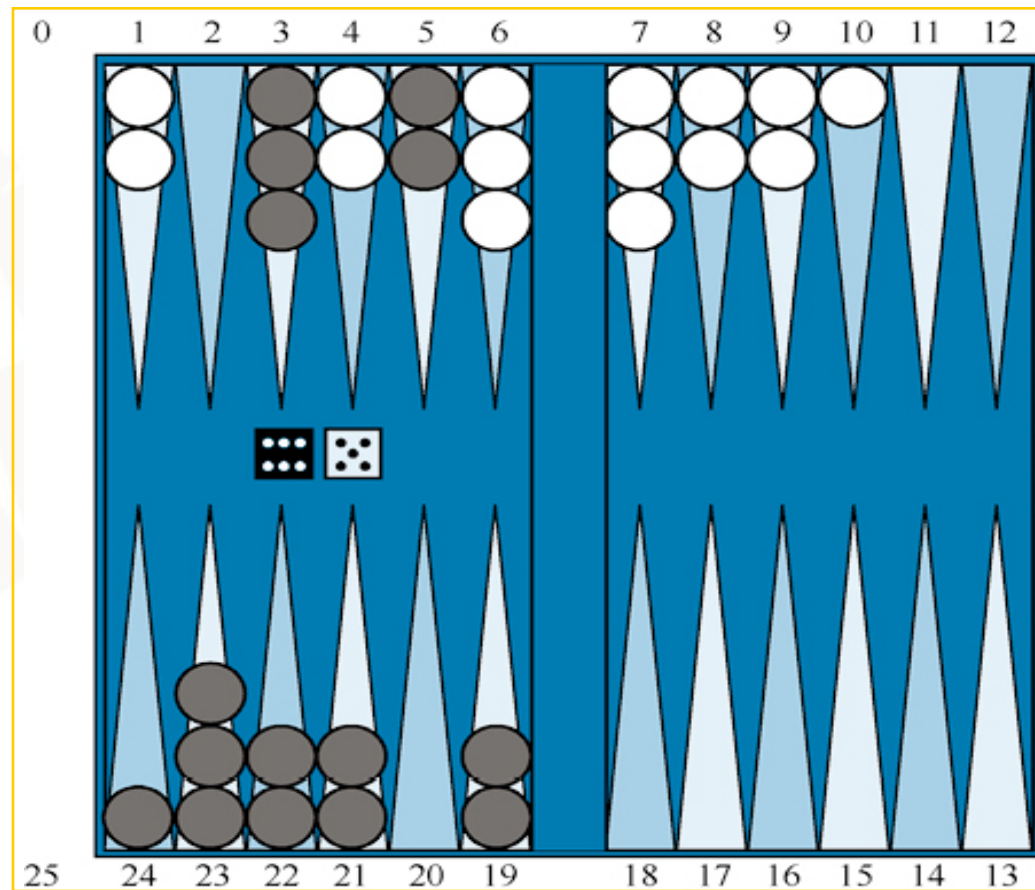
## 例：西洋双陆棋

- 双人博弈，两人轮流掷骰子、行棋
- 策略 + 运气 的随机游戏
- 黑方和白方各15枚棋子
- 根据所掷两枚骰子的点数，移动己方棋子
  - 可以按照点数分别移动两枚棋子或一枚棋子
  - 一个据点无对方棋子或只有一枚对方棋子时，己方棋子进入后，对方棋子放回起点重新开始
  - 一个据点有多枚对方棋子时，己方棋子不能进入

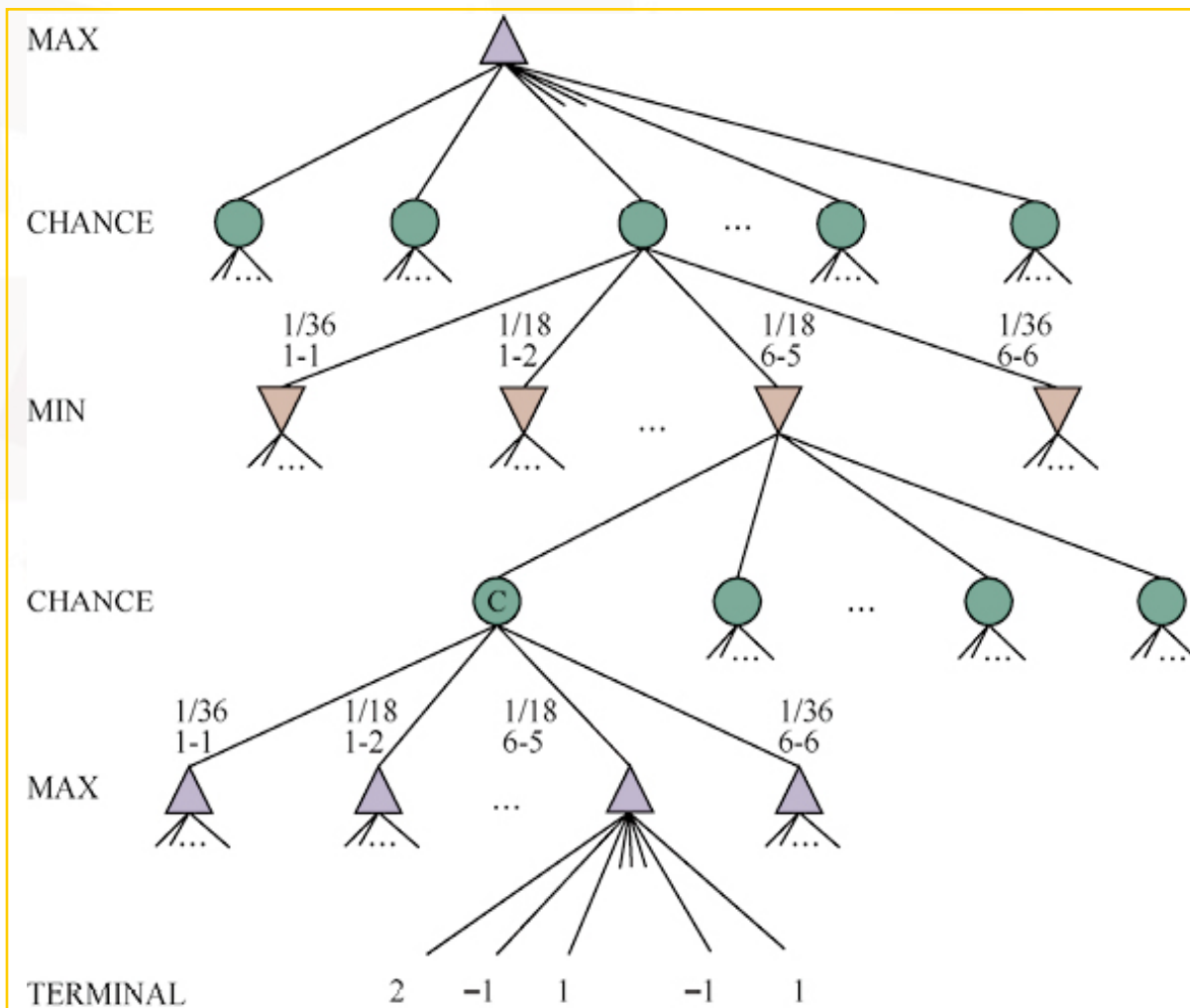


### 例：西洋双陆棋

- 当前格局，黑方掷骰子的点数为5和6
- 4种合法的移动：
  - 5-11, 5-10
  - 5-11, 19-24
  - 5-10, 10-16
  - 5-11, 11-16



- 机会节点：可能出现的情况和概率





- 期望极小极大值：对于机会节点，计算期望值，即每个机会动作的概率加权的所有结果的值之和
- $\text{ExpectMinMax}(s) =$ 
  - $\text{Utility}(s, \text{Max})$ , IF  $\text{Is-Terminal}(s)$
  - $\text{Max}_a \text{ExpectMinMax}(\text{Result}(s, a))$ , IF  $\text{To-Move}(s) = \text{Max}$
  - $\text{Min}_a \text{ExpectMinMax}(\text{Result}(s, a))$ , IF  $\text{To-Move}(s) = \text{Min}$
  - $\sum_r P(r) \text{ExpectMinMax}(\text{Result}(s, r))$ , IF  $\text{To-Move}(s) = \text{Chance}$

## 评价函数

- 截断搜索并对每个叶节点应用评价函数来近似估计期望极小极大值
- 保持叶节点值排序不变的情况下，不同的叶节点赋值改变了最佳移动
- 应返回获胜概率的正线性变换值

